

MODELE DE PRICING AUTO

Choix sur la méthode :

- la jointure nécessite de supprimer les individus sans sinistres et donne une vision incomplète de la fréquence de sinistres. Nous ferons donc la jointure après avoir ajusté le modèle fréquence (la base sévérité n'apportant que le coût de sinistres comme information additionnel).
- La sélection des variables a été faite par la méthode stepwise sur le critère de l'AIC ($AIC = -2\ell + 2k$, compromis précision du modèle / complexité.).
- Nous utilisons la validation croisée pour contrôler l'overfitting au lieu de splitter les datasets.
- Nous ajustons les GLMs avec statsmodels au lieu de scikit-learn pour avoir un plus grand choix de modèles (binomiale négative...) et parce que l'encodage est fait automatiquement avec patsy.

```
In [89]: ### Import des librairies
import sklearn
import seaborn as sns
import scipy.stats as stats
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
import warnings
import patsy
```

```
In [90]: data_path1 = "../datasets/freMTPL2freq.csv"

data_path2 = "../datasets/freMTPL2sev.csv"

df_freq = pd.read_csv(data_path1)
df_sev = pd.read_csv(data_path2, sep=";")
```

```
In [91]: pd.set_option("display.max_rows", 20)
pd.set_option("display.max_columns", 20)
pd.set_option("display.width", 100)
pd.set_option("display.max_colwidth", 50)
```

CHOIX DE LA DISTRIBUTION DE LA CIBLE Y (fréquence sinistress)

Le nombre de sinistres est une variable de comptage. Elle pourrait être modélisée par une distribution binomiale. Cependant, lorsque le nombre

d'individus est grand ($n \rightarrow +\infty$), et la probabilité sur un évènement d'observer un sinistre tend vers 0, la modélisation se fait par une loi de

Poisson. Dans notre dataset, le nombre élevé d'individus et la surreprésentation des 0 justifie la modélisation par une loi de poisson.

Le nombre élevé de 0 peut entraîner une surdispersion

($E(Y | X) < V(E(Y | X))$) ce qui ne respecte pas les hypothèses de la loi de poisson $E(Y | X) = V(Y | X) = \lambda$. Dans ce cas la loi binomiale négative lui sera préféré

```
In [92]: from scipy.stats import poisson, nbinom
# Comptages bruts
counts = df_freq['ClaimNb'].value_counts().sort_index()

# 1) Troncature et agrégation de la queue
max_k = 10
counts_trunc = counts.reindex(range(0, max_k), fill_value=0)
tail = counts[counts.index >= max_k].sum()
counts_trunc[max_k] = tail # catégorie ">= max_k"
labels = list(range(0, max_k)) + [f">={max_k}"]

# positions numériques
x = np.arange(len(labels))

# estimateur lambda
lambda_hat = df_freq['ClaimNb'].mean()

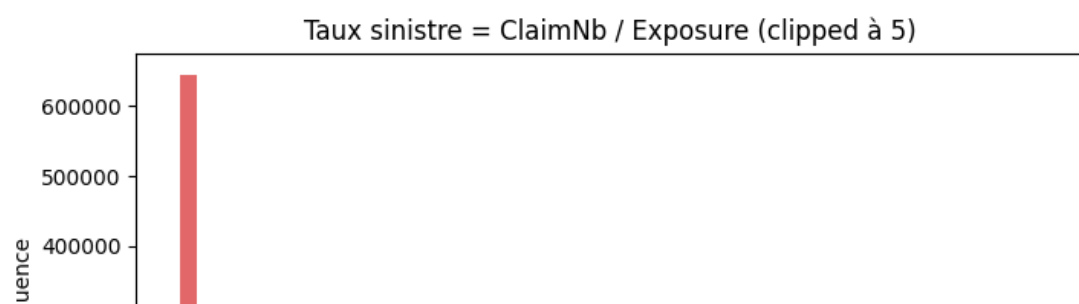
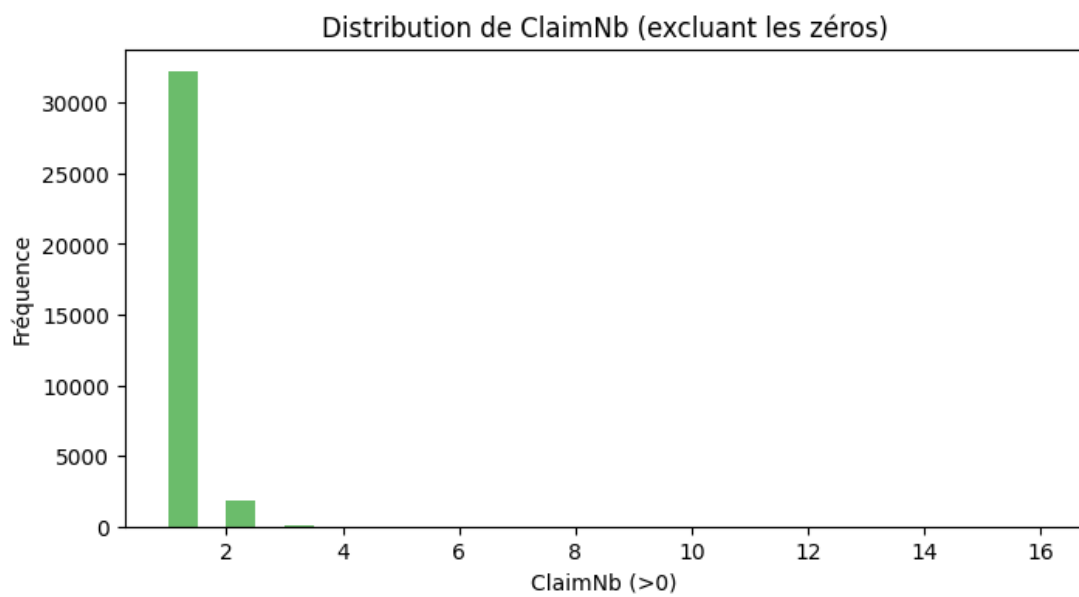
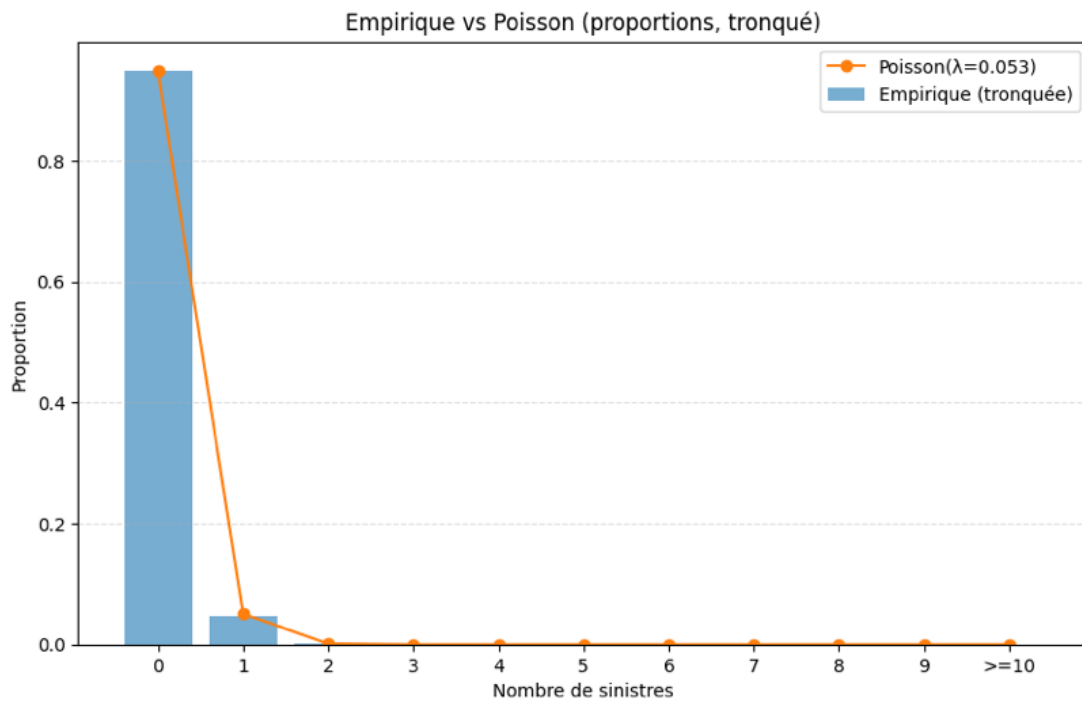
# pmf Poisson avec masse de la queue
pmf = poisson.pmf(np.arange(0, max_k), lambda_hat)
p_tail = 1 - poisson.cdf(max_k - 1, lambda_hat)
pmf_with_tail = np.append(pmf, p_tail)

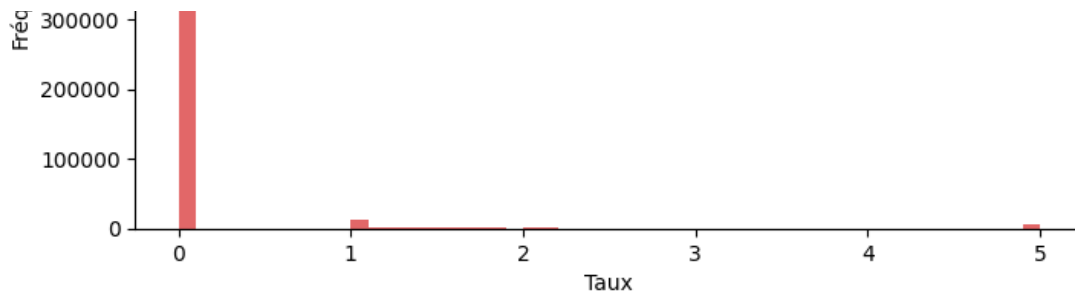
obs_prop = counts_trunc.values / counts_trunc.values.sum()
plt.figure(figsize=(10,6))
plt.bar(x, obs_prop, color='C0', alpha=0.6, label='Empirique (tronqué)')
plt.plot(x, pmf_with_tail, 'o-', color='C1', label=f'Poisson( $\lambda$ ={lambda_hat})')
plt.xticks(x, labels)
plt.title('Empirique vs Poisson (proportions, tronqué)')
plt.xlabel('Nombre de sinistres')
plt.ylabel('Proportion')
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.show()

# 3) Zoom : histogramme sans les zéros
plt.figure(figsize=(8,4))
plt.hist(df_freq.loc[df_freq['ClaimNb']>0, 'ClaimNb'], bins=30, color='C2')
plt.title('Distribution de ClaimNb (excluant les zéros)')
plt.xlabel('ClaimNb (>0)')
plt.ylabel('Fréquence')
plt.show()

# 4) Visualiser le taux ClaimNb / Exposure (fortement recommandé si
if 'Exposure' in df_freq.columns:
    rate = df_freq['ClaimNb'] / df_freq['Exposure']
    plt.figure(figsize=(8,4))
    plt.hist(rate.clip(upper=5), bins=50, color='C3', alpha=0.7) #
    plt.title('Taux sinistre = ClaimNb / Exposure (clipped à 5)')
    plt.xlabel('Taux')
```

```
plt.ylabel('Fréquence')
plt.show()
```





On va faire une première régression de poisson sur les données d'abord sans sélectionner de variables.

La fonction de lien est la fonction log et $\eta = \beta' * X$ le prédicteur linéaire

On aura $E(Y | X) = \lambda = \exp(\eta)$

avec l'offset $\phi : E(Y | X) = \lambda = \exp(\eta + \log(\phi))$

```
In [93]: # liste des colonnes de df_freq
feature_cols = [col for col in df_freq.columns]
display(feature_cols)
```

```
['IDpol',
 'ClaimNb',
 'Exposure',
 'VehPower',
 'VehAge',
 'DrivAge',
 'BonusMalus',
 'VehBrand',
 'VehGas',
 'Area',
 'Density',
 'Region']
```

L'âge du conducteur et l'âge du véhicule bien que continues, ont un effet non-linéaire sur la fréquence de sinistres. Afin de prendre en compte ces effets, des formules polynomiales par morceaux (splines) sont utilisées ici pour ces variables dans la formule du prédicteur linéaire.

```
In [94]: from patsy import bs # pour splines
dfm = df_freq.loc[df_freq["Exposure"] > 0].copy()

# Formule : caté en C(), numériques en continu (splines conseillées)
formula = (
    "ClaimNb ~ "
    "C(VehBrand) + C(VehGas) + C(Area) + C(Region) + "
    "bs(DrivAge, df=5) + bs(VehAge, df=5) + "
    "BonusMalus + VehPower + Density"
)

modelPoisson = smf.glm(
    formula=formula,
    data=dfm,
    family=sm.families.Poisson(),
    exposure=dfm["Exposure"],
)
```

```
resPoisson = modelPoisson.fit() # robuste
print(resPoisson.summary2())
```

Results: Generalized linear model

Model:	GLM	AIC:
282778.0943		
Link Function:	Log	BIC:
-8889601.0763		
Dependent Variable:	ClaimNb	Log-Likelihood:
-1.4134e+05		
Date:	2025-12-26 20:51	LL-Null:
-1.4690e+05		
No. Observations:	678013	Deviance:
2.1336e+05		
Df Model:	49	Pearson chi2:
1.59e+06		
Df Residuals:	677963	Scale:
1.0000		
Method:	IRLS	

	Coef.	Std.Err.	z	P
> z	[0.025	0.975]		
Intercept	-2.4264	0.1071	-22.6627	
0.0000 -2.6362 -2.2165				
C(VehBrand)[T.B10]	-0.0053	0.0368	-0.1442	
0.8853 -0.0775 0.0669				
C(VehBrand)[T.B11]	0.0655	0.0396	1.6542	
0.0981 -0.0121 0.1432				
C(VehBrand)[T.B12]	0.1123	0.0181	6.2063	
0.0000 0.0768 0.1477				
C(VehBrand)[T.B13]	0.0281	0.0409	0.6872	
0.4920 -0.0521 0.1084				
C(VehBrand)[T.B14]	-0.1807	0.0784	-2.3034	
0.0213 -0.3345 -0.0269				
C(VehBrand)[T.B2]	-0.0138	0.0153	-0.9018	
0.3672 -0.0437 0.0162				
C(VehBrand)[T.B3]	0.0070	0.0219	0.3195	
0.7493 -0.0359 0.0499				
C(VehBrand)[T.B4]	-0.0309	0.0298	-1.0390	
0.2988 -0.0892 0.0274				
C(VehBrand)[T.B5]	0.0638	0.0248	2.5740	
0.0101 0.0152 0.1123				
C(VehBrand)[T.B6]	-0.0459	0.0284	-1.6181	
0.1057 -0.1016 0.0097				
C(VehGas)[T.Regular]	0.0615	0.0110	5.5930	
0.0000 0.0399 0.0831				
C(Area)[T.B]	0.0545	0.0217	2.5071	
0.0122 0.0119 0.0971				
C(Area)[T.C]	0.0848	0.0180	4.7012	
0.0000 0.0495 0.1202				
C(Area)[T.D]	0.1822	0.0193	9.4203	
0.0000 0.1443 0.2201				
C(Area)[T.F]	0.2296	0.0255	8.9903	

0.0000 0.1796 0.2797			
C(Area) [T.F]	0.2639	0.0888	2.9727
0.0030 0.0899 0.4379			
C(Region) [T.Aquitaine]	-0.0602	0.0910	-0.6621
0.5079 -0.2385 0.1181			
C(Region) [T.Auvergne]	-0.2817	0.1134	-2.4841
0.0130 -0.5039 -0.0594			
C(Region) [T.Basse-Normandie]	0.0288	0.0955	0.3009
0.7635 -0.1585 0.2160			
C(Region) [T.Bourgogne]	0.0098	0.0976	0.1008
0.9197 -0.1814 0.2011			
C(Region) [T.Bretagne]	0.1086	0.0891	1.2194
0.2227 -0.0660 0.2832			
C(Region) [T.Centre]	0.0922	0.0878	1.0509
0.2933 -0.0798 0.2643			
C(Region) [T.Champagne-Ardenne]	0.1471	0.1176	1.2509
0.2110 -0.0834 0.3777			
C(Region) [T.Corse]	0.1429	0.1080	1.3233
0.1857 -0.0687 0.3545			
C(Region) [T.Franche-Comte]	-0.0788	0.1585	-0.4974
0.6189 -0.3894 0.2318			
C(Region) [T.Haute-Normandie]	-0.0443	0.1041	-0.4254
0.6706 -0.2483 0.1597			
C(Region) [T.Ile-de-France]	-0.0010	0.0889	-0.0116
0.9907 -0.1753 0.1732			
C(Region) [T.Languedoc-Roussillon]	-0.0120	0.0905	-0.1328
0.8943 -0.1895 0.1654			
C(Region) [T.Limousin]	0.2066	0.1071	1.9293
0.0537 -0.0033 0.4164			
C(Region) [T.Midi-Pyrenees]	-0.0969	0.0949	-1.0214
0.3071 -0.2828 0.0890			
C(Region) [T.Nord-Pas-de-Calais]	-0.1294	0.0900	-1.4378
0.1505 -0.3059 0.0470			
C(Region) [T.Pays-de-la-Loire]	0.0082	0.0897	0.0919
0.9268 -0.1675 0.1840			
C(Region) [T.Picardie]	0.0823	0.0991	0.8302
0.4064 -0.1119 0.2765			
C(Region) [T.Poitou-Charentes]	-0.0234	0.0928	-0.2519
0.8011 -0.2052 0.1584			
C(Region) [T.Provence-Alpes-Cotes-D'Azur]	-0.0010	0.0882	-0.0117
0.9907 -0.1739 0.1719			
C(Region) [T.Rhone-Alpes]	0.1002	0.0880	1.1397
0.2544 -0.0721 0.2726			
bs(DrivAge, df=5) [0]	-1.2023	0.0732	-16.4245
0.0000 -1.3458 -1.0589			
bs(DrivAge, df=5) [1]	-0.3587	0.0445	-8.0655
0.0000 -0.4458 -0.2715			
bs(DrivAge, df=5) [2]	-0.2523	0.0697	-3.6206
0.0003 -0.3889 -0.1157			
bs(DrivAge, df=5) [3]	-0.6855	0.0891	-7.6973
0.0000 -0.8601 -0.5110			
bs(DrivAge, df=5) [4]	0.5066	0.1748	2.8981
0.0038 0.1640 0.8492			
bs(VehAge, df=5) [0]	-1.6003	0.0280	-57.2018
0.0000 -1.6551 -1.5454			
bs(VehAge, df=5) [1]	-0.8465	0.0232	-36.4514
0.0000 -0.8920 -0.8010			
bs(VehAge, df=5) [2]	-2.5882	0.1260	-20.5421
0.0000 -2.8352 -2.3413			

bs(VehAge, df=5) [3]	-2.5177	0.7805	-3.2255
0.0013 -4.0475 -0.9878			
bs(VehAge, df=5) [4]	-1.7489	0.7970	-2.1942
0.0282 -3.3111 -0.1867			
BonusMalus	0.0228	0.0003	68.8472
0.0000 0.0221 0.0234			
VehPower	0.0091	0.0028	3.3013
0.0010 0.0037 0.0145			
Density	-0.0000	0.0000	-0.6138
0.5394 -0.0000 0.0000			

=====

=====

On va calculer l'indice de surdispersion du modèle : $[\hat{\phi} = \frac{\chi^2}{n-K}]$ où χ^2 est la statistique du khi-deux de Pearson et $n - K$ représente le nombre de degrés de liberté résiduel (nombre d'observations moins le nombre de paramètres estimés du modèle).

```
In [95]: #On va calculer l'estimateur de surdispersion
Pearson_chi2 = resPoisson.pearson_chi2
df_resid = resPoisson.df_resid
dispersion_coef = Pearson_chi2 / df_resid
print("Coefficient de dispersion :", dispersion_coef)
```

Coefficient de dispersion : 2.341843181285072

$\hat{\phi} > 1$, ce qui ne respectent pas l'hypothèse de la loi de poisson ($E(Y | X) = V(Y | X)$). On choisira une loi dont les hypothèses prennent en compte l'hétéroscédasticité tout (avec un paramètre additionnel θ) tout en étant une loi de comptage (positif, probabilité baissant quand le nombre de sinistre croît). L'hypothèse sur la variance de la loi est : $V(Y) = \mu + \frac{\mu^2}{\theta}$ Elle admet un offset et la même fonction de lien que le modèle poisson

```
In [96]: # Comptages bruts (préservé)
counts = df_freq['ClaimNb'].value_counts().sort_index()

# 1) Troncature et agrégation de la queue
max_k = 10
counts_trunc = counts.reindex(range(0, max_k), fill_value=0)
tail = counts[counts.index >= max_k].sum()
counts_trunc[max_k] = tail
labels = list(range(0, max_k)) + [f">={max_k}"]
x = np.arange(len(labels))

# estimateur de la moyenne et de la variance (population)
mu = df_freq['ClaimNb'].mean()
var = df_freq['ClaimNb'].var(ddof=0)

# estimer paramètre r (shape) et p pour scipy.stats.nbinom
if var <= mu + 1e-8:
    warnings.warn("Variance <= moyenne -> pas de surdispersion détecté")
    pmf = poisson.pmf(np.arange(0, max_k), mu)
    p_tail = 1 - poisson.cdf(max_k - 1, mu)
```

```

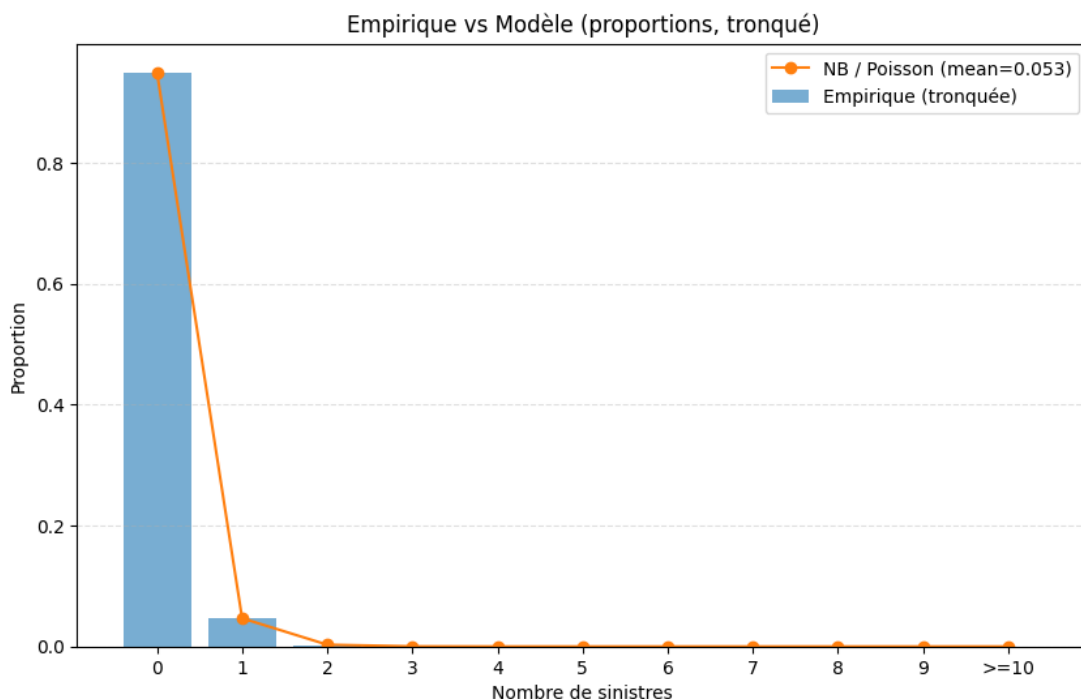
pmf_with_tail = np.append(pmf, p_tail)
print(f"Poisson utilisé: mean={mu:.4f}")
else:
    r = mu**2 / (var - mu)          # taille (shape)
    p = r / (r + mu)                # probabilité de "succès"
    # scipy nbinom parameterisation: n=r, p=p -> pmf(k) = comb(k+n-1, k) * p**p * (1-p)**n
    pmf = nbinom.pmf(np.arange(0, max_k), r, p)
    p_tail = 1 - nbinom.cdf(max_k - 1, r, p)
    pmf_with_tail = np.append(pmf, p_tail)
    print(f"NB params estimés: r={r:.4f}, p={p:.4f}, mean≈{mu:.4f}, var≈{var:.4f}")

# proportions observées
obs_prop = counts_trunc.values / counts_trunc.values.sum()

# plot
plt.figure(figsize=(10,6))
plt.bar(x, obs_prop, color='C0', alpha=0.6, label='Empirique (tronquée)')
plt.plot(x, pmf_with_tail, 'o-', color='C1', label=f'NB / Poisson (mean={mu:.4f})')
plt.xticks(x, labels)
plt.title('Empirique vs Modèle (proportions, tronqué)')
plt.xlabel('Nombre de sinistres')
plt.ylabel('Proportion')
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.show()

```

NB params estimés: r=0.6430, p=0.9235, mean≈0.0532, var≈0.0577



In [97]: #Faire un GLM comme précédemment avec une loi binomiale négative


```
dfm = df_freq.loc[df_freq["Exposure"] > 0].copy()
# Formule : caté en C(), numériques en continu (splines conseillées)
formula = (
    "ClaimNb ~ "
    "C(VehBrand) + C(VehGas) + C(Area) + C(Region) + "
    "bs(DrivAge, df=5) + bs(VehAge, df=5) + "
    "BonusMalus + VehPower + Density"
)
modelBinomNeg = smf.glm(
    formula=formula,
    data=dfm,
    family=sm.families.NegativeBinomial(alpha=1.0), # alpha à ajuster
    exposure=dfm["Exposure"],
)
resBinomNeg = modelBinomNeg.fit() # robuste
print(resBinomNeg.summary())
```

Generalized Linear Model Regression Results

```
=====
Dep. Variable:          ClaimNb    No. Observations:
678013
Model:                  GLM        Df Residuals:
677963
Model Family:          NegativeBinomial    Df Model:
49
Link Function:          Log        Scale:
1.0000
Method:                 IRLS        Log-Likelihood:
1.4094e+05
Date:                   Fri, 26 Dec 2025    Deviance:
1.8539e+05
Time:                   20:51:43    Pearson chi2:
1.53e+06
No. Iterations:          9    Pseudo R-squ. (CS):
0.01562
Covariance Type:        nonrobust
=====
```

				coef	std err	
z	P> z	[0.025	0.975]			
Intercept				-2.3997	0.113	-2
1.296	0.000	-2.621	-2.179			
C(VehBrand) [T.B10]				-0.0062	0.038	-
0.162	0.871	-0.081	0.069			
C(VehBrand) [T.B11]				0.0639	0.041	
1.552	0.121	-0.017	0.145			
C(VehBrand) [T.B12]				0.1155	0.019	
6.148	0.000	0.079	0.152			
C(VehBrand) [T.B13]				0.0239	0.043	
0.562	0.574	-0.060	0.107			
C(VehBrand) [T.B14]				-0.1858	0.081	-
2.297	0.022	-0.344	-0.027			
C(VehBrand) [T.B2]				-0.0142	0.016	-
0.898	0.369	-0.045	0.017			
C(VehBrand) [T.B3]				0.0062	0.023	
0.273	0.785	-0.038	0.051			
C(VehBrand) [T.B4]				0.0202	0.021	

C(VehBrand) [T.B4]				-0.0502	0.051	-
0.980	0.327	-0.091	0.030			
C(VehBrand) [T.B5]				0.0668	0.026	
2.591	0.010	0.016	0.117			
C(VehBrand) [T.B6]				-0.0454	0.029	-
1.543	0.123	-0.103	0.012			
C(VehGas) [T.Regular]				0.0637	0.011	
5.577	0.000	0.041	0.086			
C(Area) [T.B]				0.0554	0.022	
2.462	0.014	0.011	0.099			
C(Area) [T.C]				0.0855	0.019	
4.579	0.000	0.049	0.122			
C(Area) [T.D]				0.1845	0.020	
9.201	0.000	0.145	0.224			
C(Area) [T.E]				0.2333	0.027	
8.788	0.000	0.181	0.285			
C(Area) [T.F]				0.2721	0.093	
2.939	0.003	0.091	0.454			
C(Region) [T.Aquitaine]				-0.0568	0.095	-
0.597	0.551	-0.243	0.130			
C(Region) [T.Auvergne]				-0.2784	0.118	-
2.363	0.018	-0.509	-0.048			
C(Region) [T.Basse-Normandie]				0.0375	0.100	
0.375	0.707	-0.158	0.233			
C(Region) [T.Bourgogne]				0.0213	0.102	
0.209	0.835	-0.179	0.221			
C(Region) [T.Bretagne]				0.1194	0.093	
1.280	0.200	-0.063	0.302			
C(Region) [T.Centre]				0.1020	0.092	
1.110	0.267	-0.078	0.282			
C(Region) [T.Champagne-Ardenne]				0.1646	0.123	
1.343	0.179	-0.076	0.405			
C(Region) [T.Corse]				0.1542	0.113	
1.367	0.172	-0.067	0.375			
C(Region) [T.Franche-Comte]				-0.0702	0.164	-
0.427	0.670	-0.393	0.252			
C(Region) [T.Haute-Normandie]				-0.0319	0.108	-
0.294	0.769	-0.244	0.181			
C(Region) [T.Ile-de-France]				0.0078	0.093	
0.084	0.933	-0.175	0.190			
C(Region) [T.Languedoc-Roussillon]				-2.056e-05	0.095	-
0.000	1.000	-0.186	0.186			
C(Region) [T.Limousin]				0.2179	0.112	
1.949	0.051	-0.001	0.437			
C(Region) [T.Midi-Pyrenees]				-0.0890	0.099	-
0.898	0.369	-0.283	0.105			
C(Region) [T.Nord-Pas-de-Calais]				-0.1218	0.094	-
1.293	0.196	-0.306	0.063			
C(Region) [T.Pays-de-la-Loire]				0.0139	0.094	
0.148	0.882	-0.170	0.198			
C(Region) [T.Picardie]				0.0947	0.104	
0.914	0.361	-0.108	0.298			
C(Region) [T.Poitou-Charentes]				-0.0166	0.097	-
0.171	0.864	-0.207	0.174			
C(Region) [T.Provence-Alpes-Cotes-D'Azur]				0.0058	0.092	
0.063	0.950	-0.175	0.187			
C(Region) [T.Rhone-Alpes]				0.1095	0.092	
1.189	0.234	-0.071	0.290			
bs(DrivAge, df=5) [0]				-1.2320	0.077	-1
5.070	0.000	1.202	1.001			

5.979	0.000	-1.585	-1.081			
bs(DrivAge, df=5) [1]				-0.3633	0.047	-
7.701	0.000	-0.456	-0.271			
bs(DrivAge, df=5) [2]				-0.2499	0.073	-
3.404	0.001	-0.394	-0.106			
bs(DrivAge, df=5) [3]				-0.7124	0.093	-
7.624	0.000	-0.895	-0.529			
bs(DrivAge, df=5) [4]				0.5418	0.184	
2.938	0.003	0.180	0.903			
bs(VehAge, df=5) [0]				-1.6760	0.029	-5
7.548	0.000	-1.733	-1.619			
bs(VehAge, df=5) [1]				-0.8967	0.024	-3
7.127	0.000	-0.944	-0.849			
bs(VehAge, df=5) [2]				-2.6797	0.130	-2
0.637	0.000	-2.934	-2.425			
bs(VehAge, df=5) [3]				-2.4942	0.798	-
3.125	0.002	-4.058	-0.930			
bs(VehAge, df=5) [4]				-1.8697	0.834	-
2.241	0.025	-3.505	-0.234			
BonusMalus				0.0233	0.000	6
4.417	0.000	0.023	0.024			
VehPower				0.0090	0.003	
3.137	0.002	0.003	0.015			
Density				-2.349e-06	3.84e-06	-
0.612	0.540	-9.87e-06	5.17e-06			
=====						
=====						

Adéquation du modèle

Déviance :

Ecart entre la log-vraisemblance modèle saturé et modèle ajusté (la plus basse possible)

Test de Wald (Z) :

Significativité des variables Compare à l'hypothèse H_0 : "La valeur de l'estimateur de la variable est 0" On veut rejeter le test (si $\alpha = 5$ la P-value doit être inférieur à 0,05 ou 0 dans l'IC)

Robustesse du modèle

Résidus de Pearson standardisé :

Vérification visuelle : pseudo-linéarité (résidus doivent être répartis de manière aléatoire autour de 0), hétéroscédasticité (bande quasi-horizontale autour de 0 sans effet entonnoir)

Performance du modèle

Performance du modele

Courbe de Lorenz:

Mesure la capacité du modèle à ordonner les individus en fonction du niveau de risque (on reste proche de la première bissectrice)

```
In [98]: #calculer la déviance du modèle saturé et comparer à la déviance de
# Déviance : Ecart entre la log-vraisemblance modèle saturé et mod
deviance_saturated = 0 # par définition
deviance_model = resBinomNeg.deviance
print(f"Déviance du modèle binomial négatif: {deviance_model:.4f}")
```

Déviance du modèle binomial négatif: 185385.1872

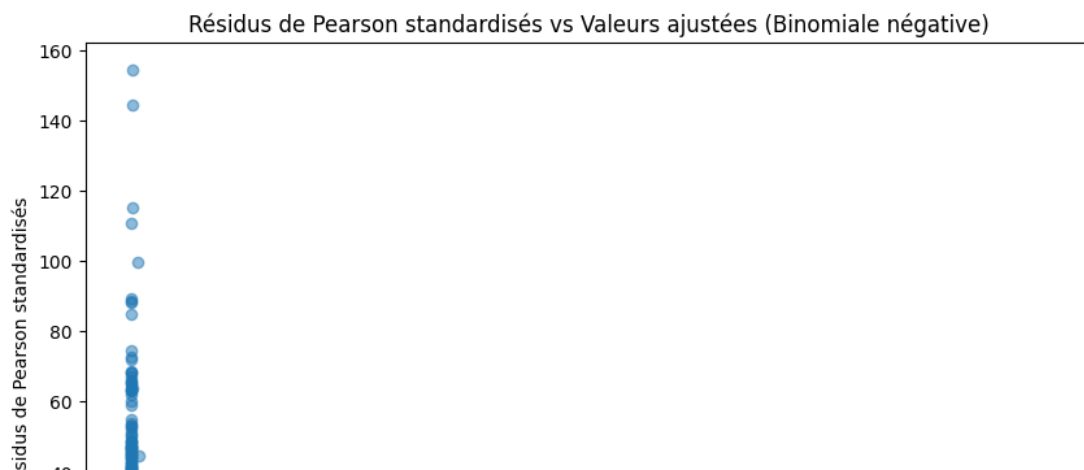
```
In [99]: #Donner la liste et le nombre des variables significatives au seuil
significant_vars = resBinomNeg.pvalues[resBinomNeg.pvalues < 0.05]
print("Variables significatives au seuil 5% :", significant_vars)
print("Nombre de variables significatives au seuil 5% :", len(signi
```

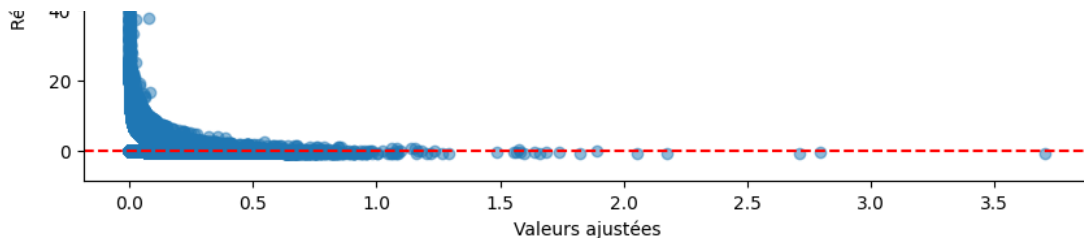
Variables significatives au seuil 5% : ['Intercept', 'C(VehBrand)[T.B12]', 'C(VehBrand)[T.B14]', 'C(VehBrand)[T.B5]', 'C(VehGas)[T.Regular]', 'C(Area)[T.B]', 'C(Area)[T.C]', 'C(Area)[T.D]', 'C(Area)[T.E]', 'C(Area)[T.F]', 'C(Region)[T.Auvergne]', 'bs(DrivAge, df=5)[0]', 'bs(DrivAge, df=5)[1]', 'bs(DrivAge, df=5)[2]', 'bs(DrivAge, df=5)[3]', 'bs(DrivAge, df=5)[4]', 'bs(VehAge, df=5)[0]', 'bs(VehAge, df=5)[1]', 'bs(VehAge, df=5)[2]', 'bs(VehAge, df=5)[3]', 'bs(VehAge, df=5)[4]', 'BonusMalus', 'VehPower']

Nombre de variables significatives au seuil 5% : 23

```
In [100]: #Graphe des Résidus de Pearson standardisé sur la binomiale négative

pearson_resid = resBinomNeg.resid_pearson
fitted_vals = resBinomNeg.fittedvalues
plt.figure(figsize=(10,6))
plt.scatter(fitted_vals, pearson_resid, alpha=0.5)
plt.axhline(0, color='red', linestyle='--')
plt.title('Résidus de Pearson standardisés vs Valeurs ajustées (Binomiale négative)')
plt.xlabel('Valeurs ajustées')
plt.ylabel('Résidus de Pearson standardisés')
plt.show()
```





Le graphique des résidus de Pearson standardisés présente énormément d'effectifs à zéro une majorité de points concentrée autour de zéro, indiquant une bonne adéquation globale du modèle. Quelques valeurs extrêmes apparaissent pour des observations à faible fréquence ajustée, phénomène attendu dans les modèles de fréquence automobile en raison de la rareté des sinistres (forte proportion de zéros). Ces résidus élevés ne traduisent pas un défaut de spécification, mais la nature discrète et surdispersée des données.

Pour une distribution de nombre de sinistre X par exemple, la courbe de Lorenz est tracée en associant à chaque nombre de sinistre x un point dont les

coordonnées sont $F(x)$ en abscisse et $G(x)$ en ordonnée. • $F(x)$ représente la proportion des individus ayant moins de x sinistres. • $G(x)$ représente la proportion de la masse totale M des sinistres correspondant à ces individus (Masse des sinistres $< x$ / Masse totale M)

Cette courbe est toujours située en dessous de la première bissectrice, car $F(x) > G(x)$

Nous pondérons les $F(x)$ par l'exposure

La courbe de Lorenz permet d'apprécier la concentration du risque au sein d'un portefeuille.

Pour une variable de comptage X représentant le nombre de sinistres, elle associe à chaque proportion cumulée d'exposition $F(x)$ la proportion cumulée de sinistres observés $G(x)$:

$$F(x) = \frac{\sum_{i: X_i \leq x} \text{Exposure}_i}{\sum_i \text{Exposure}_i}, \quad G(x) = \frac{\sum_{i: X_i \leq x} X_i}{\sum_i X_i}.$$

La courbe de Lorenz est toujours située en dessous de la première bissectrice, c'est-à-dire : $F(x) \geq G(x)$, $\forall x$ ce qui traduit le fait qu'une minorité d'assurés concentre la majorité des sinistres.

Dans le cadre d'un modèle de fréquence, la courbe est tracée après tri des contrats par risque prédictif croissant $\hat{\mu}_i$.

Elle permet ainsi de mesurer la capacité discriminante du modèle :

un écart marqué à la diagonale indique une bonne différenciation des risques, tandis qu'une courbe proche de la bissectrice traduit un faible pouvoir de discrimination.

In [101]...

```
# ==== Courbe de Lorenz pondérée & Indice de Gini (fréquence) ====
# Prérequis : 'resBinomNeg' = résultat d'un GLM statsmodels avec ex
```

```

# - Abscisse : exposition cumulée (pondération)
# - Ordonnée : sinistres observés cumulés (comptes, sans repondérat.
# - Tri : par taux prédit (comptes prédits / exposition)
# - Gini (tri croissant) :  $G = 1 - 2 * AUC$ 

def lorenz_gini_deciles(res_glm, y_col="ClaimNb", w_col="Exposure",
    # 1) Récupération alignée (mêmes lignes, même ordre que le fit)
    df_fit = res_glm.model.data.frame.copy()
    y = df_fit[y_col].astype(float).to_numpy()
    w = df_fit[w_col].astype(float).to_numpy()
    mu_counts = np.asarray(res_glm.fittedvalues, dtype=float) # E[...]

    # Nettoyage
    m = np.isfinite(y) & np.isfinite(w) & np.isfinite(mu_counts) &
    y, w, mu_counts = y[m], w[m], mu_counts[m]

    # 2) Sanity check (fréquences moyennes)
    freq_obs = y.sum() / w.sum()
    freq_pred = mu_counts.sum() / w.sum()
    print(f"Fréquence moyenne observée : {freq_obs:.6f}")
    print(f"Fréquence moyenne prédite : {freq_pred:.6f}")

    # 3) Construction Lorenz : trier par TAUX prédit = comptes / ex
    mu_rate = mu_counts / w
    order = np.argsort(mu_rate) # tri croissant du ris
    y, w, mu_rate = y[order], w[order], mu_rate[order]

    # Abscisse (exposition cumulée) & Ordonnée (sinistres cumulés)
    x = np.cumsum(w) / w.sum()
    Y = np.cumsum(y) / y.sum()
    # Forcer (0,0) et (1,1)
    x = np.concatenate(([0.0], x, [1.0]))
    Y = np.concatenate(([0.0], Y, [1.0]))

    # 4) Indice de Gini (tri croissant -> courbe sous la diagonale)
    auc = np.trapezoid(Y, x) # numpy >= 2.0
    gini = abs(1.0 - 2.0 * auc)
    print(f"Indice de Gini pondéré (fréquence) : {gini:.3f}")

    # 5) Tracé
    plt.figure(figsize=(7,7))
    plt.plot(x, Y, label="Lorenz pondérée (modèle)")
    plt.plot([0,1], [0,1], "r--", label="Égalité parfaite")
    plt.xlabel("Proportion cumulée d'exposition (tri par risque cro
    plt.ylabel("Proportion cumulée des sinistres observés")
    plt.title(plot_title or "Courbe de Lorenz pondérée - Modèle de
    plt.legend(); plt.grid(True); plt.show()

    # 6) Table par déciles (discrimination + calibration simple)
    df_rank = pd.DataFrame({"y": y, "w": w, "mu_rate": mu_rate})
    df_rank["decile"] = pd.qcut(df_rank["mu_rate"], 10, labels=False)
    tab = df_rank.groupby("decile").apply(
        lambda g: pd.Series({
            "exposition": g["w"].sum(),
            "sinistres": g["y"].sum(),
            "freq_obs": g["y"].sum() / g["w"].sum(),
            "freq_pred": np.average(g["mu_rate"], weights=g["w"])

```

```

    })
  )
  f_mean = freq_obs
  tab["lift_obs"] = tab["freq_obs"] / f_mean
  tab["lift_pred"] = tab["freq_pred"] / f_mean
  print("\nTable par déciles (tri croissant du risque) :")
  print(tab.round(4))

  return {"gini": gini, "x": x, "Y": Y, "deciles": tab}

résultats = lorenz_gini_deciles(resBinomNeg, y_col="ClaimNb", w_col=
                                plot_title="Courbe de Lorenz pondérée

```

Fréquence moyenne observée : 0.100703

Fréquence moyenne prédite : 0.102357

Indice de Gini pondéré (fréquence) : 0.281

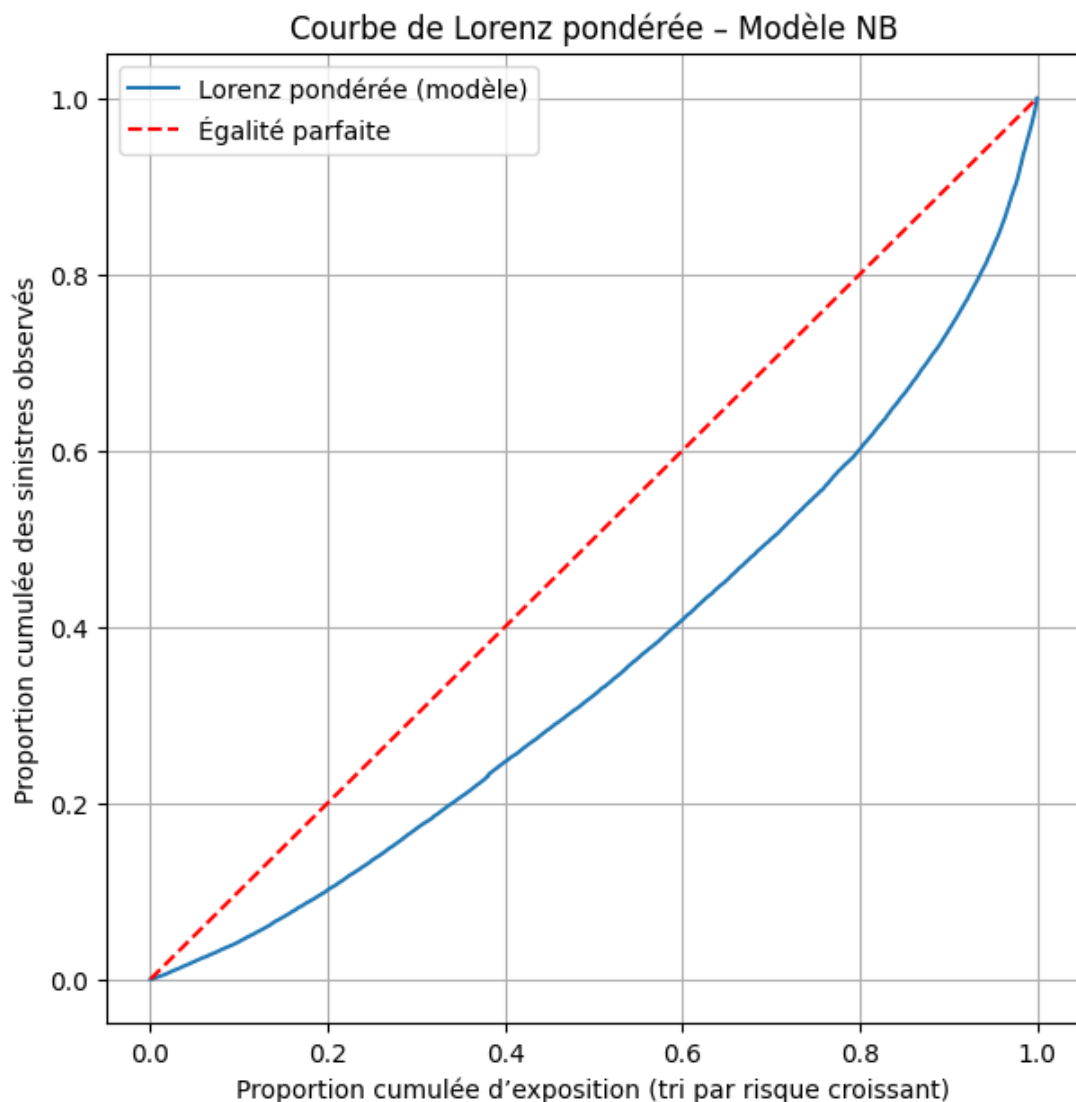


Table par déciles (tri croissant du risque) :

	exposition	sinistres	freq_obs	freq_pred	lift_obs	lift_p
red decile						
1	41211.1675	1826.0	0.0443	0.0513	0.4400	0.5
095						
2	40731.4545	2517.0	0.0618	0.0631	0.6136	0.6
264						
3	40274.6707	2886.0	0.0717	0.0702	0.7116	0.6
974						
4	39724.7084	3088.0	0.0777	0.0767	0.7719	0.7
613						
5	39404.6934	3196.0	0.0811	0.0829	0.8054	0.8
237						
6	38523.2454	3485.0	0.0905	0.0898	0.8983	0.8
920						
7	36366.2207	3601.0	0.0990	0.0988	0.9833	0.9
816						
8	32542.1021	3939.0	0.1210	0.1164	1.2020	1.1
555						
9	28224.1075	4643.0	0.1645	0.1724	1.6336	1.7
119						
10	21497.0753	6921.0	0.3220	0.3331	3.1970	3.3
078						

```
/var/folders/61/k5ckz_dn2hs4k4v4pjww68p80000gn/T/ipykernel_66303/1871136082.py:56: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.
```

```
tab = df_rank.groupby("decile").apply(
```

Le modèle de fréquence ajusté sous loi binomiale négative présente une bonne adéquation globale : la fréquence moyenne prédite (0.102) est très proche de la fréquence observée (0.101), ce qui traduit une calibration correcte du modèle. La courbe de Lorenz montre une courbure marquée, traduisant une différenciation effective des risques. L'indice de Gini pondéré, égal à 0.281, indique un pouvoir discriminant satisfaisant.

Nous faisons maintenant par validation croisé l'indice de GINI et la courbe de Lorenz pour vérifier qu'il n'y a pas d'overfitting

```
In [102... # ===== Validation croisée k-fold : courbes de Lorenz 00F + Gini =====
```



```

from sklearn.model_selection import KFold

def kfold_lorenz_oof(res_glm, y_col="ClaimNb", w_col="Exposure",
                    k=5, shuffle=True, random_state=42, plot=True,
                    ****
    Refit le GLM (même formula & family) sur k-1 folds et prédit sur
    Calcule et trace la courbe de Lorenz OOF + Gini par fold et global
    ****

    # Récupère la data & les méta du modèle existant (sans rien rec
    df_all = res_glm.model.data.frame.copy()
    formula = getattr(res_glm.model, "formula", None)
    family = res_glm.family
    if formula is None:
        raise ValueError("Le modèle original doit avoir été ajusté ")

    # Structures OOF
    oof_counts = pd.Series(index=df_all.index, dtype=float)

    # KFold scikit-learn
    kf = KFold(n_splits=k, shuffle=shuffle, random_state=random_state)
    gini_folds = []

    for i, (tr_idx, va_idx) in enumerate(kf.split(df_all), start=1):
        df_tr = df_all.iloc[tr_idx]
        df_va = df_all.iloc[va_idx]

        # Refit GLM (fonction existante)
        model = smf.glm(formula=formula, data=df_tr, family=family,
                        res_fold = model.fit())

        # Prédiction COMPTE sur le fold de validation
        mu_va_counts = res_fold.predict(df_va, exposure=df_va[w_col])
        oof_counts.iloc[va_idx] = mu_va_counts.to_numpy(float)

        # Lorenz & Gini du fold (utilise pandas/numpy)
        df_va_rank = df_va.assign(mu_rate = mu_va_counts / df_va[w_col])
        xv = df_va_rank[w_col].cumsum() / df_va_rank[w_col].sum()
        yv = df_va_rank[y_col].cumsum() / df_va_rank[y_col].sum()
        xv = pd.concat([pd.Series([0.0]), xv, pd.Series([1.0])]).to_numpy()
        yv = pd.concat([pd.Series([0.0]), yv, pd.Series([1.0])]).to_numpy()
        try:
            auc_v = np.trapezoid(yv, xv)
        except AttributeError:
            auc_v = np.trapz(yv, xv)
        gini_v = abs(1.0 - 2.0*auc_v)
        gini_folds.append(gini_v)
        print(f"[CV] Fold {i}/{k} : Gini OOF = {gini_v:.3f}")

    # Courbe & Gini OOF "global"
    df_oof = df_all.assign(mu_rate = oof_counts / df_all[w_col])
    xo = df_oof[w_col].cumsum() / df_oof[w_col].sum()
    yo = df_oof[y_col].cumsum() / df_oof[y_col].sum()
    xo = pd.concat([pd.Series([0.0]), xo, pd.Series([1.0])]).to_numpy()
    yo = pd.concat([pd.Series([0.0]), yo, pd.Series([1.0])]).to_numpy()
    try:
        auc_o = np.trapezoid(yo, xo)
    except AttributeError:
        auc_o = np.trapz(yo, xo)

```

```

gini_oof = abs(1.0 - 2.0*auc_o)

print(f"\n[CV] Gini OOF moyen (± écart-type) : {np.mean(gini_fo
print(f"[CV] Gini OOF global (OOF recollé) : {gini_oof:.3f}")

# Tracé OOF
if plot:
    plt.figure(figsize=(7,7))
    plt.plot(xo, yo, label=f"Lorenz OOF ({k}-fold)")
    if overlay_insample and 'résultats' in globals():
        plt.plot(résultats["x"], résultats["Y"], label="Lorenz
    plt.plot([0,1], [0,1], "r--", label="Égalité parfaite")
    plt.xlabel("Proportion cumulée d'exposition (tri par risque
    plt.ylabel("Proportion cumulée des sinistres observés")
    ttl = f"Courbe de Lorenz OOF - Validation croisée {k}-fold"
    if overlay_insample: ttl += " (overlay in-sample)"
    plt.title(ttl)
    plt.legend(); plt.grid(True); plt.show()

    return {
        "gini_folds": gini_folds,
        "gini_oof": gini_oof,
        "x_oof": xo,
        "y_oof": yo,
        "oof_counts": oof_counts
    }

# ===== Lancement de la validation croisée (OOF) =====
cv = kfold_lorenz_oof(
    resBinomNeg,
    y_col="ClaimNb",
    w_col="Exposure",
    k=5, shuffle=True, random_state=42,
    plot=True,
    overlay_insample=True # met à True si 'résultats' (in-sample)
)

```

```

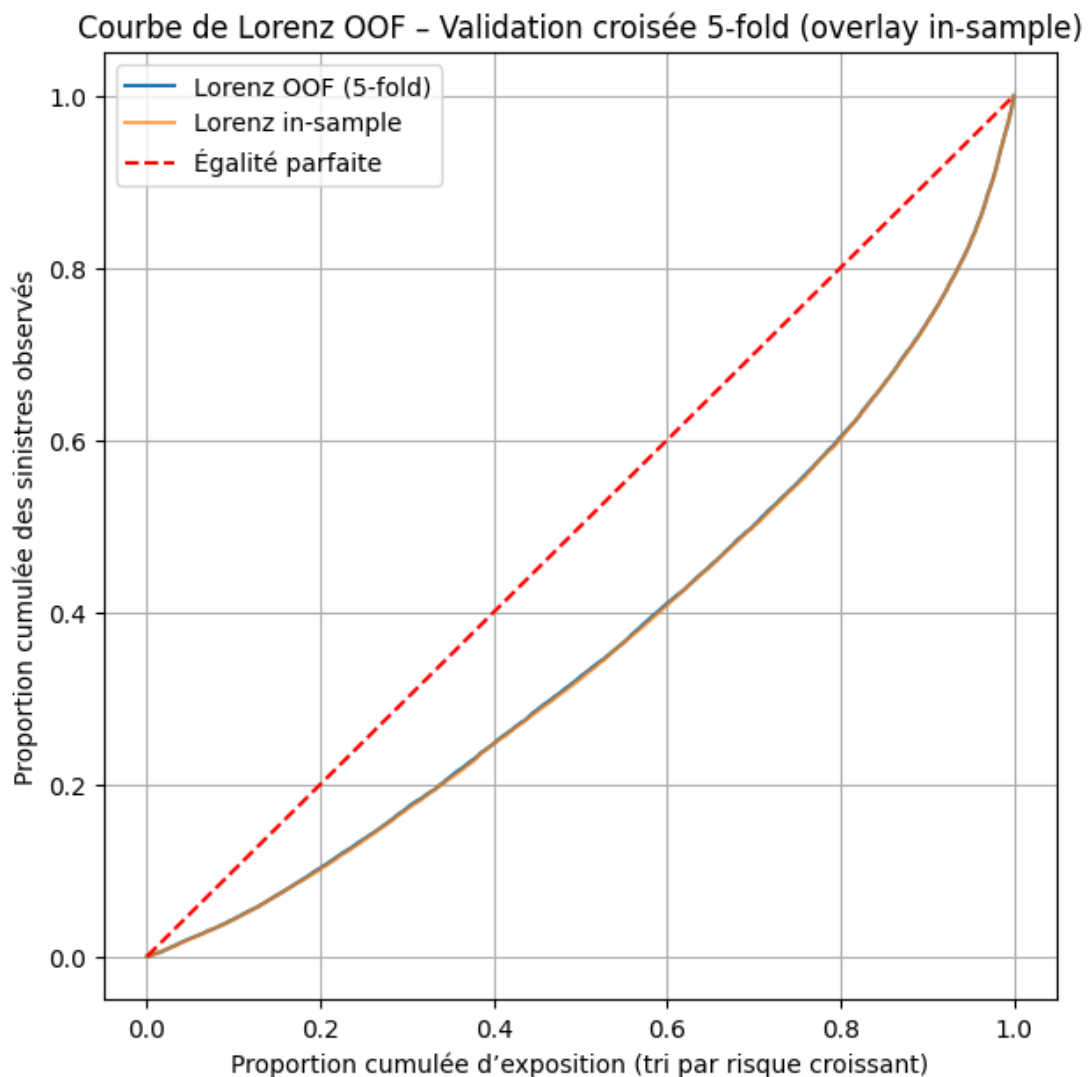
[CV] Fold 1/5 : Gini OOF = 0.271
[CV] Fold 2/5 : Gini OOF = 0.268
[CV] Fold 3/5 : Gini OOF = 0.281
[CV] Fold 4/5 : Gini OOF = 0.282
[CV] Fold 5/5 : Gini OOF = 0.292

```

```

[CV] Gini OOF moyen (± écart-type) : 0.279 ± 0.010
[CV] Gini OOF global (OOF recollé) : 0.279

```



La validation croisée 5-fold donne un indice de Gini moyen de 0.279 (+/- 0.010), très proche du Gini in-sample (0.281). Cela indique une bonne capacité de généralisation du modèle binomial négatif, sans signe d'overfitting. La superposition des courbes de Lorenz in-sample et OOF (en validation croisée) confirme visuellement la stabilité du modèle.

Toutes les variables disponibles ont été conservées, la validation croisée n'indiquant pas de sur-apprentissage.

```
In [103... # 1) Coefficients (table propre)
coefs = pd.DataFrame({
    "param": resBinomNeg.params.index,
    "beta": resBinomNeg.params.values
})
print(coefs)

# 2) Fréquence prédite (taux par unité d'exposition) et nombre attendu
df_fit = resBinomNeg.model.data.frame.copy()
w = df_fit["Exposure"].astype(float).to_numpy()
mu_counts = np.asarray(resBinomNeg.fittedvalues, dtype=float) # E[
lambda_rate = mu_counts / w # fr
```

```
print("Fréquence moyenne prédite :", lambda_rate.mean())
print("Comptes moyens prédits :", mu_counts.mean())
```

```

           param      beta
0      Intercept -2.399662
1  C(VehBrand)[T.B10] -0.006186
2  C(VehBrand)[T.B11]  0.063945
3  C(VehBrand)[T.B12]  0.115481
4  C(VehBrand)[T.B13]  0.023909
..
45 bs(VehAge, df=5)[3] -2.494199
46 bs(VehAge, df=5)[4] -1.869707
47      BonusMalus  0.023295
48      VehPower  0.008991
49      Density -0.000002

```

[50 rows x 2 columns]

```
Fréquence moyenne prédite : 0.11624746693885092
Comptes moyens prédits    : 0.054121443164726196
```

MODELE SEVERITE - LOI GAMMA

```

In [104... df_sev.head()
df_sev.info()
df_sev
#Créer un dataset 'df_cout' qui joint df_freq et df_sev sur IDpol p
df_cout = pd.merge(df_freq, df_sev, on="IDpol", how="inner")
df_cout = df_cout.drop(columns=["ClaimNb", "IDpol"])
df_cout
#Vérifier on devrait avoir 11 colonnes
print(df_cout.shape) # (, 11) On a moins de de lignes à cause des
display(df_cout)
df_cout.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26639 entries, 0 to 26638
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   IDpol            26639 non-null  int64
1   ClaimAmount      26639 non-null  float64
dtypes: float64(1), int64(1)
memory usage: 416.4 KB
(26444, 11)

```

Exposure VehPower VehAge DrivAge BonusMalus VehBrand VehG

0	0.75	7	1	61	50	B12	Regul
1	0.14	12	5	50	60	B12	Dies
2	0.14	4	0	36	85	B12	Regul
3	0.62	10	0	51	100	B12	Regul
4	0.62	10	0	51	100	B12	Regul
...	...	...	...	...	...	...	...
26439	0.14	7	2	51	50	B12	Dies
26440	0.17	4	0	35	51	B12	Regul
26441	0.17	15	3	36	50	B12	Regul
26442	0.17	15	3	36	50	B12	Regul
26443	0.13	7	10	39	50	B4	Regul

26444 rows × 11 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26444 entries, 0 to 26443
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Exposure        26444 non-null  float64
1   VehPower        26444 non-null  int64
2   VehAge          26444 non-null  int64
3   DrivAge         26444 non-null  int64
4   BonusMalus      26444 non-null  int64
5   VehBrand        26444 non-null  object
6   VehGas          26444 non-null  object
7   Area            26444 non-null  object
8   Density         26444 non-null  int64
9   Region          26444 non-null  object
10  ClaimAmount     26444 non-null  float64
dtypes: float64(2), int64(5), object(4)
memory usage: 2.2+ MB
```

In [105... *#distribution de la variable ClaimAmount*

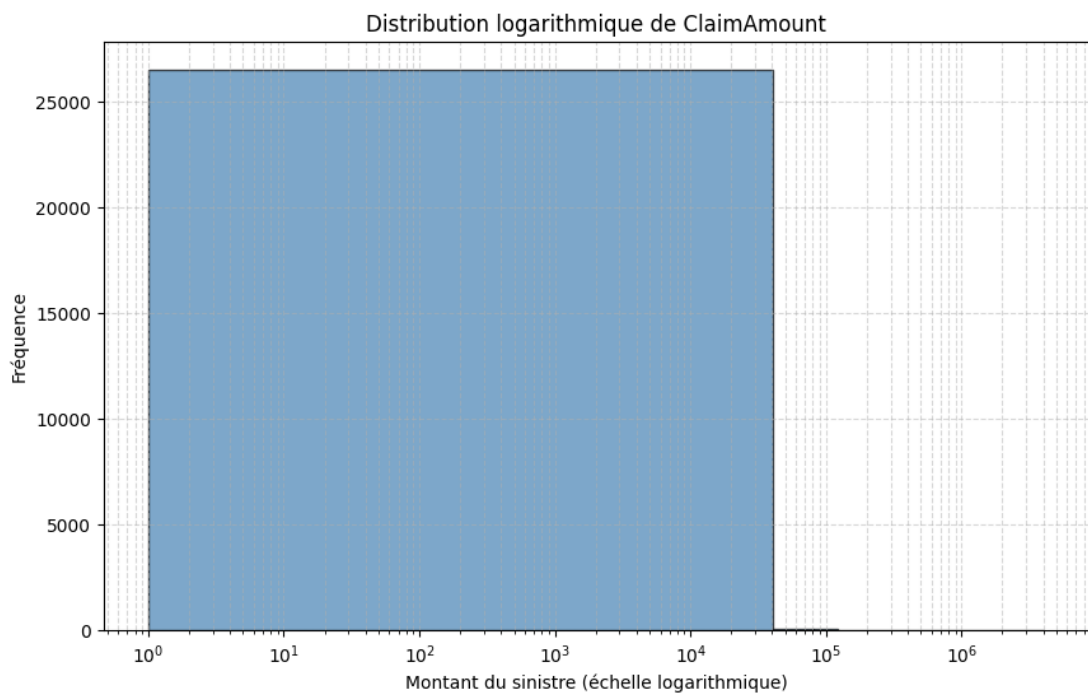
```
plt.figure(figsize=(10,6))

plt.hist(df_sev["ClaimAmount"], bins=100, color='steelblue', edgeco
plt.xscale("log")
plt.xlabel("Montant du sinistre (échelle logarithmique)")
plt.ylabel("Fréquence")
plt.title("Distribution logarithmique de ClaimAmount")
```

```
plt.grid(True, which="both", ls="--", alpha=0.5)
plt.show()

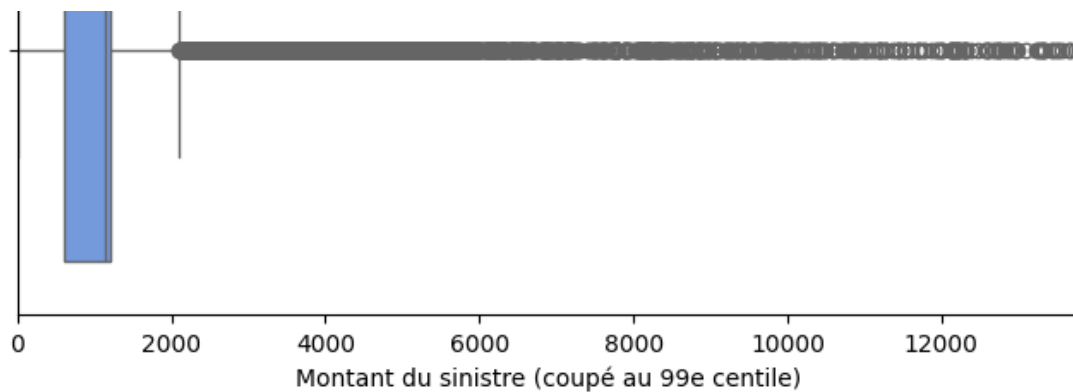
plt.figure(figsize=(8,4))
sns.boxplot(x=df_sev["ClaimAmount"], color='cornflowerblue')
plt.xlim(0, df_sev["ClaimAmount"].quantile(0.99)) # on coupe les 1%
plt.xlabel("Montant du sinistre (coupé au 99e centile)")
plt.title("Boxplot des montants de sinistres (hors valeurs extrêmes)")
plt.show()

#Faire un résumé statistique de la variable ClaimAmount (avec les données)
print(df_cout['ClaimAmount'].describe().round(2))
```



Boxplot des montants de sinistres (hors valeurs extrêmes)





```
count      26444.00
mean       2077.53
std        29267.68
min         1.00
25%         602.00
50%        1128.12
75%        1204.00
max       4075400.56
Name: ClaimAmount, dtype: float64
```

CHOIX DE LA DISTRIBUTION DE LA CIBLE ClaimAmount

La variable ClaimAmount présente une distribution extrêmement asymétrique à droite. La majorité des sinistres sont de faible montant, tandis qu'un petit nombre de cas atteignent plusieurs centaines de milliers, voire plusieurs millions d'euros. Cette forte hétérogénéité justifie l'utilisation d'une loi Gamma (ou log-normale) pour modéliser la sévérité, permettant de représenter la variance croissante avec la moyenne.

Les 2 lois sont utilisées pour modéliser les coûts des sinistres (variables continues et positives). La loi Gamma est adaptée aux branches présentant avec une grande majorité de sinistres peu coûteux et des sinistres graves rares. La loi log-normale, moins fréquemment utilisée, modélise des portefeuilles avec des sinistres graves plus importants (queue de distribution plus épaisse que la loi gamma).

Nous allons tester l'adéquation de la variable cible ClaimAmount à une loi gamma par un qq-plot (droite de Henry)

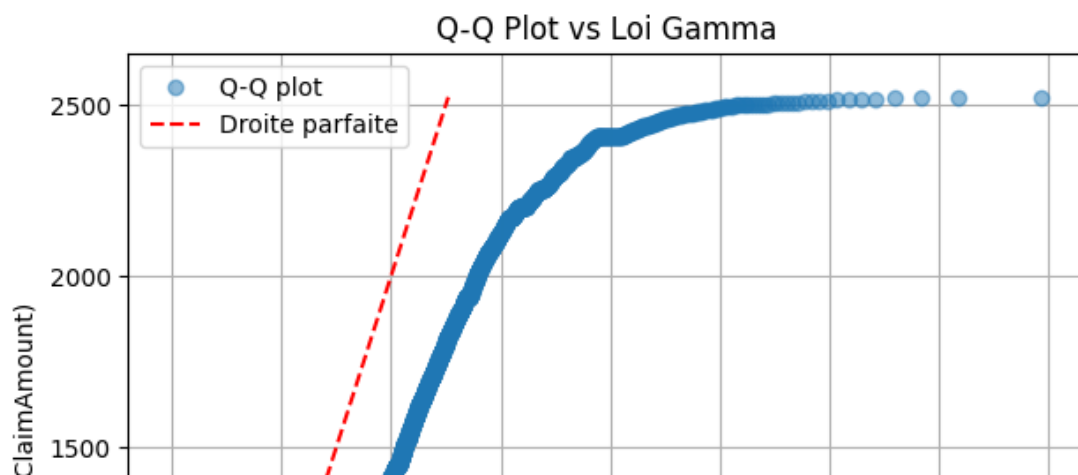
```
In [106... #tester l'adéquation de la variable cible ClaimAmount à une loi gamma
# et par un test d'adéquation du khi II (df_cout['ClaimAmount'])

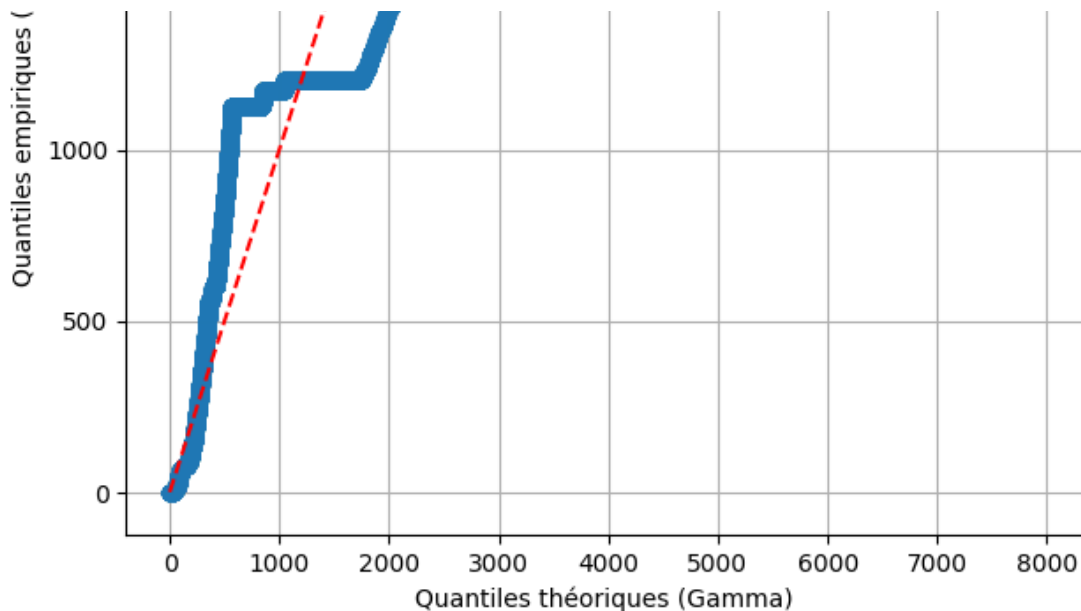
#Données
data = df_cout['ClaimAmount'].dropna()
data = data[data > 0] #sécurité si 0
# Coupe au 90e centile
cutoff = np.percentile(data, 90)
data_trimmed = data[data <= cutoff]
#=====
# Q-Q Plot : Loi Gamma
# =====
# Ajustement des paramètres Gamma (shape, loc=0, scale)
```

```

gamma_params = stats.gamma.fit(data_trimmed, floc=0)
shape, loc, scale = gamma_params
# Quantiles théoriques (n points uniformes → transformés par la loi gamma)
n = len(data_trimmed)
probs = (np.arange(1, n+1) - 0.5) / n
theoretical_q_gamma = stats.gamma.ppf(probs, a=shape, loc=loc, scale=scale)
# Quantiles empiriques
empirical_q = np.sort(data_trimmed)
# Tracé
plt.figure(figsize=(7,7))
plt.plot(theoretical_q_gamma, empirical_q, 'o', alpha=0.5, label='Q-Q plot')
plt.plot([empirical_q.min(), empirical_q.max()], [empirical_q.min(), empirical_q.max()], 'r--', label='Droite parfaite')
plt.xlabel('Quantiles théoriques (Gamma)')
plt.ylabel('Quantiles empiriques (ClaimAmount)')
plt.title('Q-Q Plot vs Loi Gamma')
plt.legend();
plt.grid(True)
plt.show()

```





Malgré une bonne lisibilité après exclusion des sinistres extrêmes (coupure au 90e centile), la loi Gamma tend à surestimer la queue de la distribution des sinistres. L'ajustement est acceptable dans les faibles et moyens sinistres, ce qui reste compatible avec son usage courant pour modéliser la sévérité dans des portefeuilles dominés par de petits sinistres.

On va effectuer une première régression Gamma sur les données, sans sélection préalable de variables explicatives.

La fonction de lien choisie est la fonction logarithme. La fonction canonique est la fonction inverse, mais la fonction logarithme est préférée en non-vie car :

- Il garantit que les primes (ou les coûts moyens) prédites μ ne sont jamais négatives.
- Chaque facteur de risque a un effet proportionnel sur la prime totale Le prédicteur linéaire est noté $\eta = \beta' X$.

Le modèle est défini par : $E(Y | X) = \mu = \exp(\eta)$

L'hypothèse sur la variance est la suivante : $\text{Var}(Y | X) = \alpha \mu^2$, où α est le paramètre de dispersion estimé.

```
In [107.. # Préparer et nettoyer les données
dfm_sev = df_cout.copy()
mask = (
    dfm_sev["ClaimAmount"].notna() &
    np.isfinite(dfm_sev["ClaimAmount"]) &
    (dfm_sev["ClaimAmount"] > 0)
)
dfm_sev = dfm_sev.loc[mask].copy()
print("Après nettoyage, n =", len(dfm_sev))
print(dfm_sev["ClaimAmount"].describe().round(2))

# Winsorisation pour éviter overflow dans exp(link)
cap = dfm_sev["ClaimAmount"].quantile(0.999) # ajuster à 0.99 si n
```

```

print("Capping ClaimAmount au quantile 99.9% =", cap)
dfm_sev["ClaimAmount_clipped"] = dfm_sev["ClaimAmount"].clip(upper=cap)

# Formule (utilise la variable clipped)
formula_sev = (
    "ClaimAmount_clipped ~ "
    "C(VehBrand) + C(VehGas) + C(Area) + C(Region) + "
    "bs(DrivAge, df=5) + bs(VehAge, df=5) + "
    "BonusMalus + VehPower + Density"
)

# GLM Gamma lien log – sans exposure ni offset
modelGamma = smf.glm(
    formula=formula_sev,
    data=dfm_sev,
    family=sm.families.Gamma(link=sm.families.links.Log()),
)

# Fit avec maxiter et affichage minimal
resGamma = modelGamma.fit(maxiter=200, disp=False)
print(resGamma.summary2())

```

Après nettoyage, n = 26444

```

count      26444.00
mean        2077.53
std         29267.68
min           1.00
25%          602.00
50%         1128.12
75%         1204.00
max        4075400.56

```

Name: ClaimAmount, dtype: float64

Capping ClaimAmount au quantile 99.9% = 138454.99838001

Results: Generalized linear model

```

=====
=====

```

Model:	GLM	AIC:
494323.7751		
Link Function:	Log	BIC:
-227985.3394		
Dependent Variable:	ClaimAmount_clipped	Log-Likeliho
d: -2.4711e+05		
Date:	2025-12-26 20:52	LL-Null:
-2.4716e+05		
No. Observations:	26444	Deviance:
40779.		
Df Model:	49	Pearson chi2:
2.71e+05		
Df Residuals:	26394	Scale:
10.255		
Method:	IRLS	

```

-----
-----

```

	Coef.	Std.Err.	z	P>
z [0.025 0.975]				

Intercept	7.5145	0.4084	18.3978	0.
0000 6.7140 8.3150				

C(VehBrand)[T.B10]	0.1266	0.1264	1.0018	0.
3164 -0.1211 0.3742				
C(VehBrand)[T.B11]	0.2249	0.1331	1.6901	0.
0910 -0.0359 0.4858				
C(VehBrand)[T.B12]	0.0110	0.0716	0.1535	0.
8780 -0.1293 0.1512				
C(VehBrand)[T.B13]	0.0836	0.1446	0.5778	0.
5634 -0.1999 0.3670				
C(VehBrand)[T.B14]	-0.0648	0.2829	-0.2292	0.
8187 -0.6193 0.4896				
C(VehBrand)[T.B2]	0.0015	0.0550	0.0270	0.
9784 -0.1063 0.1092				
C(VehBrand)[T.B3]	0.0200	0.0764	0.2615	0.
7937 -0.1297 0.1696				
C(VehBrand)[T.B4]	0.1221	0.1046	1.1675	0.
2430 -0.0829 0.3270				
C(VehBrand)[T.B5]	-0.1052	0.0878	-1.1987	0.
2306 -0.2773 0.0668				
C(VehBrand)[T.B6]	-0.1651	0.0988	-1.6712	0.
0947 -0.3588 0.0285				
C(VehGas)[T.Regular]	0.0067	0.0410	0.1631	0.
8705 -0.0737 0.0871				
C(Area)[T.B]	-0.1209	0.0844	-1.4314	0.
1523 -0.2863 0.0446				
C(Area)[T.C]	-0.1394	0.0696	-2.0019	0.
0453 -0.2759 -0.0029				
C(Area)[T.D]	-0.0598	0.0738	-0.8104	0.
4177 -0.2044 0.0848				
C(Area)[T.E]	-0.1236	0.0962	-1.2855	0.
1986 -0.3121 0.0649				
C(Area)[T.F]	-0.4361	0.3341	-1.3052	0.
1918 -1.0909 0.2188				
C(Region)[T.Aquitaine]	0.2827	0.3485	0.8111	0.
4173 -0.4004 0.9658				
C(Region)[T.Auvergne]	-0.1317	0.4303	-0.3060	0.
7596 -0.9749 0.7116				
C(Region)[T.Basse-Normandie]	0.4789	0.3670	1.3049	0.
1919 -0.2404 1.1983				
C(Region)[T.Bourgogne]	0.3247	0.3769	0.8615	0.
3889 -0.4140 1.0633				
C(Region)[T.Bretagne]	0.3737	0.3429	1.0899	0.
2757 -0.2983 1.0457				
C(Region)[T.Centre]	0.3684	0.3377	1.0910	0.
2753 -0.2934 1.0302				
C(Region)[T.Champagne-Ardenne]	0.8436	0.4960	1.7009	0.
0890 -0.1285 1.8157				
C(Region)[T.Corse]	0.6843	0.4366	1.5676	0.
1170 -0.1713 1.5400				
C(Region)[T.Franche-Comte]	0.2182	0.6184	0.3528	0.
7242 -0.9939 1.4303				
C(Region)[T.Haute-Normandie]	0.1676	0.3980	0.4212	0.
6736 -0.6124 0.9477				
C(Region)[T.Ile-de-France]	0.2360	0.3428	0.6884	0.
4912 -0.4359 0.9080				
C(Region)[T.Languedoc-Roussillon]	0.0539	0.3488	0.1544	0.
8773 -0.6299 0.7376				
C(Region)[T.Limousin]	0.0752	0.4061	0.1851	0.
8531 -0.7208 0.8712				
C(Region)[T.Midi-Pyrenees]	0.0214	0.3738	0.0572	0.

9544 -0.7113 0.7541				
C(Region)[T.Nord-Pas-de-Calais]	0.2702	0.3452	0.7827	0.
4338 -0.4064 0.9469				
C(Region)[T.Pays-de-la-Loire]	0.2033	0.3440	0.5910	0.
5545 -0.4710 0.8776				
C(Region)[T.Picardie]	0.3648	0.3801	0.9597	0.
3372 -0.3802 1.1099				
C(Region)[T.Poitou-Charentes]	0.1999	0.3537	0.5652	0.
5719 -0.4933 0.8931				
C(Region)[T.Provence-Alpes-Cotes-D'Azur]	0.3316	0.3392	0.9775	0.
3283 -0.3333 0.9964				
C(Region)[T.Rhone-Alpes]	0.2565	0.3380	0.7588	0.
4480 -0.4060 0.9190				
bs(DrivAge, df=5)[0]	-0.7015	0.2563	-2.7373	0.
0062 -1.2038 -0.1992				
bs(DrivAge, df=5)[1]	-0.3568	0.1570	-2.2731	0.
0230 -0.6645 -0.0492				
bs(DrivAge, df=5)[2]	-0.9148	0.2529	-3.6166	0.
0003 -1.4105 -0.4190				
bs(DrivAge, df=5)[3]	0.3959	0.3475	1.1395	0.
2545 -0.2851 1.0769				
bs(DrivAge, df=5)[4]	-0.9396	0.7090	-1.3253	0.
1851 -2.3291 0.4500				
bs(VehAge, df=5)[0]	0.0394	0.1408	0.2801	0.
7794 -0.2365 0.3154				
bs(VehAge, df=5)[1]	-0.0480	0.1041	-0.4610	0.
6448 -0.2520 0.1560				
bs(VehAge, df=5)[2]	-0.0675	0.4616	-0.1463	0.
8837 -0.9723 0.8372				
bs(VehAge, df=5)[3]	-2.4924	2.8649	-0.8700	0.
3843 -8.1074 3.1226				
bs(VehAge, df=5)[4]	0.5208	2.8167	0.1849	0.
8533 -4.9998 6.0414				
BonusMalus	0.0024	0.0012	1.8976	0.
0577 -0.0001 0.0048				
VehPower	0.0156	0.0105	1.4860	0.
1373 -0.0050 0.0362				
Density	0.0000	0.0000	0.5080	0.
6115 -0.0000 0.0000				

=====

=====

Le modèle étant peu performant nous l'améliorant en faisant une sélection de variable par la méthode stepwise sur le critère de l'AIC

```
In [ ]: # Nettoyage des données (sans utiliser Exposure)
dfm_sev = df_cout.copy()
mask = (
    dfm_sev["ClaimAmount"].notna() &
    np.isfinite(dfm_sev["ClaimAmount"]) &
    (dfm_sev["ClaimAmount"] > 0)
)
dfm_sev = dfm_sev.loc[mask].copy()

# Winsorisation
cap = dfm_sev["ClaimAmount"].quantile(0.999)
dfm_sev["ClaimAmount_clipped"] = dfm_sev["ClaimAmount"].clip(upper=cap)
```

```

# Variables candidates
variables_cat = ["C(VehBrand)", "C(VehGas)", "C(Area)", "C(Region)"]
variables_spl = ["bs(DrivAge, df=5)", "bs(VehAge, df=5)"]
variables_cont = ["BonusMalus", "VehPower", "Density"]
all_terms = variables_cat + variables_spl + variables_cont

# Stepwise selection avec AIC (sans exposure)
def stepwise_selection(data, response, candidate_terms):
    included = []
    while True:
        changed = False
        candidates = list(set(candidate_terms) - set(included))
        scores = []
        for term in candidates:
            try:
                formula = f"{response} ~ {' + '.join(included + [term])}"
                model = smf.glm(formula=formula, data=data,
                               family=sm.families.Gamma(link=sm.families.Gamma.link_func))
                result = model.fit(dispatch=False, maxiter=100)
                scores.append((result.aic, term))
            except Exception:
                continue
        if not scores:
            break
        scores.sort()
        best_new_score, best_term = scores[0]

        # AIC du modèle courant (sans le nouveau terme) – on doit faire un modèle sans le terme
        if included:
            try:
                cur_formula = f"{response} ~ {' + '.join(included)}"
                cur_model = smf.glm(formula=cur_formula, data=data,
                                    family=sm.families.Gamma(link=sm.families.Gamma.link_func))
                cur_res = cur_model.fit(dispatch=False, maxiter=100)
                current_score = cur_res.aic
            except Exception:
                current_score = np.inf
        else:
            current_score = np.inf

        if best_new_score < current_score:
            included.append(best_term)
            changed = True
        if not changed:
            break
    return included

# Exécution forward simple (alternative / compatible)
selected = []
candidates = all_terms.copy()
max_steps = len(candidates)

for _ in range(max_steps):
    best_term = None
    best_aic = np.inf
    for term in candidates:
        if term in selected:
            continue
        trv:

```

```

        formula = f"ClaimAmount_clipped ~ {' + '.join(selected)
        res = smf.glm(formula=formula, data=dfm_sev,
                      family=sm.families.Gamma(link=sm.families
        if res.aic < best_aic:
            best_aic = res.aic
            best_term = term
    except Exception:
        continue

    if best_term is None:
        break

    # AIC du modèle courant (sans le nouveau terme)
    if selected:
        try:
            cur_res = smf.glm(formula=f"ClaimAmount_clipped ~ {' +
                              data=dfm_sev,
                              family=sm.families.Gamma(link=sm.fami
            cur_aic = cur_res.aic
        except Exception:
            cur_aic = np.inf
    else:
        cur_aic = np.inf

    # Ajouter le terme si amélioration d'AIC
    if best_aic + 1e-8 < cur_aic:
        selected.append(best_term)
        candidates.remove(best_term)
    else:
        break

print("Variables sélectionnées :")
print(selected)
# Formule finale
final_formula = f"ClaimAmount_clipped ~ {' + '.join(selected)}"
print("Formule finale :", final_formula)

```

On ajuste le modèle final

```

In [ ]: # Préparation des données : nettoyage robuste (sans Exposure)
dfm_sev = df_cout.copy()
mask = (
    dfm_sev["ClaimAmount"].notna() &
    np.isfinite(dfm_sev["ClaimAmount"]) &
    (dfm_sev["ClaimAmount"] > 0)
)
dfm_sev = dfm_sev.loc[mask].copy()
print("Après nettoyage, n =", len(dfm_sev))
print(dfm_sev["ClaimAmount"].describe().round(2))

# Winsorisation (pour éviter explosion numérique)
cap = dfm_sev["ClaimAmount"].quantile(0.999)
print("Capping ClaimAmount au quantile 99.9% =", cap)
dfm_sev["ClaimAmount_clipped"] = dfm_sev["ClaimAmount"].clip(upper=

# Formule avec les variables retenues par stepwise
formula_finale = (
    "ClaimAmount_clipped ~ C(Region) + bs(DrivAge, df=5) + C(Veh

```

```

)

# Modèle GLM Gamma avec lien log – sans exposure
modelGamma_final = smf.glm(
  formula=formula_finale,
  data=dfm_sev,
  family=sm.families.Gamma(link=sm.families.links.Log())
)

# Ajustement
resGamma_final = modelGamma_final.fit(maxiter=100, disp=False)
print(resGamma_final.summary2())

```

Après nettoyage, n = 26444

```

count      26444.00
mean        2077.53
std         29267.68
min          1.00
25%          602.00
50%         1128.12
75%         1204.00
max        4075400.56

```

Name: ClaimAmount, dtype: float64

Capping ClaimAmount au quantile 99.9% = 138454.99838001

Results: Generalized linear model

```

=====
=====

```

Model:	GLM	AIC:
493901.5004		
Link Function:	Log	BIC:
-227972.7041		
Dependent Variable:	ClaimAmount_clipped	Log-Likeliho
d: -2.4691e+05		
Date:	2025-12-26 20:25	LL-Null:
-2.4696e+05		
No. Observations:	26444	Deviance:
40873.		
Df Model:	41	Pearson chi2:
2.68e+05		
Df Residuals:	26402	Scale:
10.146		
Method:	IRLS	

```

-----
-----

```

	Coef.	Std.Err.	z	P>
z [0.025 0.975]				

Intercept	7.8010	0.3688	21.1532	0.
0000 7.0782 8.5238				
C(Region)[T.Aquitaine]	0.2663	0.3466	0.7684	0.
4423 -0.4130 0.9456				
C(Region)[T.Auvergne]	-0.1368	0.4278	-0.3198	0.
7491 -0.9754 0.7017				
C(Region)[T.Basse-Normandie]	0.4635	0.3650	1.2699	0.
2041 -0.2519 1.1789				
C(Region)[T.Bourgogne]	0.3255	0.3748	0.8684	0.
3852 -0.4091 1.0601				
C(Region)[T.Bretagne]	0.3607	0.3410	1.0579	0.
2001 -0.2076 1.0780				

2901 -0.3070 1.0200				
C(Region)[T.Centre]	0.3522	0.3358	1.0491	0.
2941 -0.3058 1.0103				
C(Region)[T.Champagne-Ardenne]	0.8282	0.4932	1.6791	0.
0931 -0.1385 1.7950				
C(Region)[T.Corse]	0.7190	0.4342	1.6559	0.
0977 -0.1320 1.5700				
C(Region)[T.Franche-Comte]	0.2098	0.6150	0.3412	0.
7330 -0.9956 1.4153				
C(Region)[T.Haute-Normandie]	0.1536	0.3958	0.3881	0.
6980 -0.6221 0.9293				
C(Region)[T.Ile-de-France]	0.2359	0.3409	0.6919	0.
4890 -0.4323 0.9040				
C(Region)[T.Languedoc-Roussillon]	0.0377	0.3469	0.1087	0.
9134 -0.6421 0.7175				
C(Region)[T.Limousin]	0.0581	0.4039	0.1438	0.
8857 -0.7335 0.8496				
C(Region)[T.Midi-Pyrenees]	0.0119	0.3718	0.0321	0.
9744 -0.7167 0.7406				
C(Region)[T.Nord-Pas-de-Calais]	0.2490	0.3433	0.7253	0.
4683 -0.4239 0.9219				
C(Region)[T.Pays-de-la-Loire]	0.1880	0.3421	0.5496	0.
5826 -0.4825 0.8585				
C(Region)[T.Picardie]	0.3455	0.3780	0.9141	0.
3607 -0.3954 1.0864				
C(Region)[T.Poitou-Charentes]	0.1869	0.3517	0.5313	0.
5952 -0.5025 0.8762				
C(Region)[T.Provence-Alpes-Cotes-D'Azur]	0.3177	0.3373	0.9421	0.
3462 -0.3433 0.9788				
C(Region)[T.Rhone-Alpes]	0.2481	0.3361	0.7380	0.
4605 -0.4107 0.9069				
C(VehBrand)[T.B10]	0.1570	0.1217	1.2900	0.
1971 -0.0816 0.3956				
C(VehBrand)[T.B11]	0.2632	0.1297	2.0297	0.
0424 0.0090 0.5174				
C(VehBrand)[T.B12]	0.0605	0.0656	0.9223	0.
3564 -0.0681 0.1892				
C(VehBrand)[T.B13]	0.1212	0.1427	0.8493	0.
3957 -0.1585 0.4008				
C(VehBrand)[T.B14]	-0.0436	0.2813	-0.1552	0.
8767 -0.5949 0.5076				
C(VehBrand)[T.B2]	0.0012	0.0546	0.0214	0.
9829 -0.1058 0.1081				
C(VehBrand)[T.B3]	0.0353	0.0756	0.4668	0.
6406 -0.1129 0.1835				
C(VehBrand)[T.B4]	0.1307	0.1037	1.2604	0.
2075 -0.0725 0.3339				
C(VehBrand)[T.B5]	-0.0998	0.0872	-1.1434	0.
2529 -0.2708 0.0712				
C(VehBrand)[T.B6]	-0.1556	0.0980	-1.5872	0.
1125 -0.3477 0.0365				
C(Area)[T.B]	-0.1137	0.0840	-1.3542	0.
1757 -0.2783 0.0509				
C(Area)[T.C]	-0.1313	0.0692	-1.8982	0.
0577 -0.2668 0.0043				
C(Area)[T.D]	-0.0473	0.0731	-0.6472	0.
5175 -0.1905 0.0959				
C(Area)[T.E]	-0.1140	0.0953	-1.1965	0.
2315 -0.3008 0.0727				
C(Area)[T.F]	-0.1207	0.2222	-1.2665	0.


```

C(Area) [1,1] -0.4207 0.5522 -1.2005 0.
2053 -1.0718 0.2304
bs(DrivAge, df=5)[0] -0.7321 0.2530 -2.8933 0.
0038 -1.2281 -0.2362
bs(DrivAge, df=5)[1] -0.4472 0.1440 -3.1054 0.
0019 -0.7295 -0.1650
bs(DrivAge, df=5)[2] -1.0177 0.2441 -4.1698 0.
0000 -1.4961 -0.5393
bs(DrivAge, df=5)[3] 0.3026 0.3395 0.8912 0.
3728 -0.3629 0.9680
bs(DrivAge, df=5)[4] -1.0480 0.7031 -1.4905 0.
1361 -2.4260 0.3301
Density 0.0000 0.0000 0.5351 0.
5926 -0.0000 0.0000
=====
=====

```

Analyse des résidus

```

In [ ]: # --- Résidus de déviance et Pearson standardisés ---
infl = resGamma_final.get_influence()
hii = infl.hat_matrix_diag

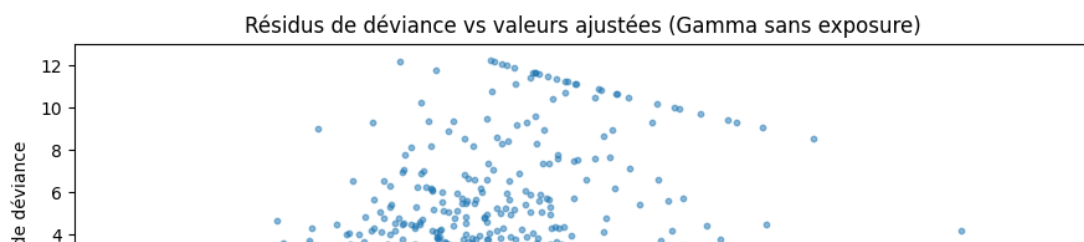
resid_dev = resGamma_final.resid_deviance
resid_pear = resGamma_final.resid_pearson
pear_std = resid_pear / np.sqrt(1 - np.clip(hii, 0, 0.99999)) # s

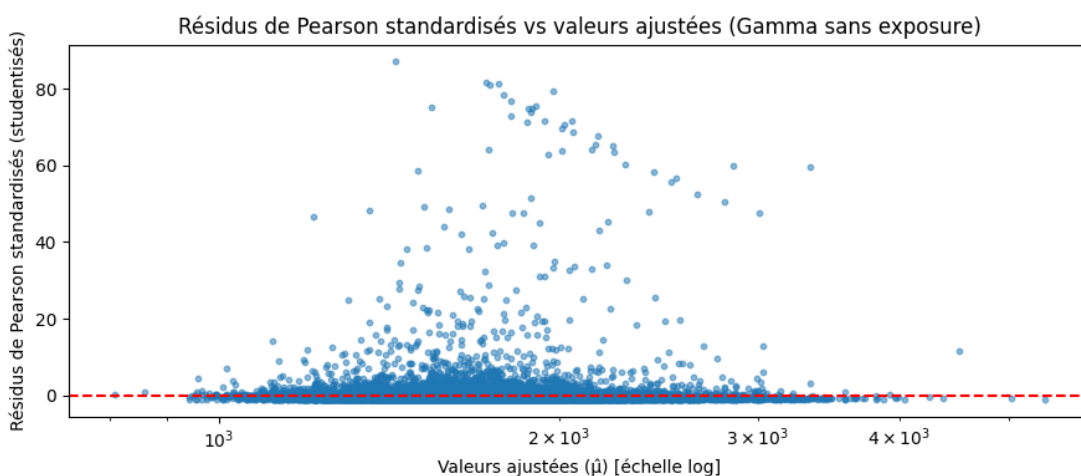
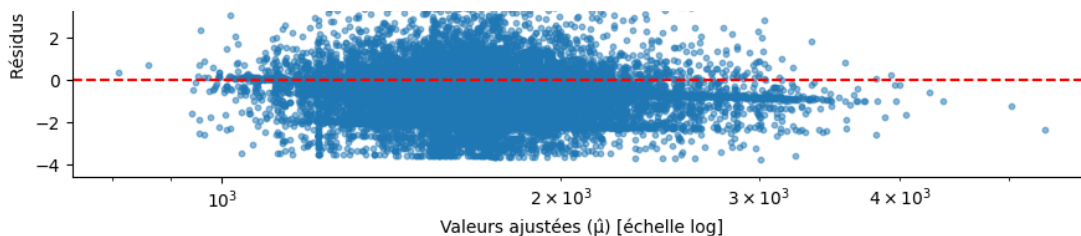
fitted = resGamma_final.fittedvalues
x_for_plot = np.maximum(fitted, np.finfo(float).eps) # éviter log(0)

# Résidus de déviance vs fitted
plt.figure(figsize=(9,4))
plt.scatter(x_for_plot, resid_dev, s=10, alpha=0.5)
plt.axhline(0, color='r', ls='--')
plt.xscale('log')
plt.title("Résidus de déviance vs valeurs ajustées (Gamma sans expos")
plt.xlabel("Valeurs ajustées ( $\hat{\mu}$ ) [échelle log]")
plt.ylabel("Résidus de déviance")
plt.tight_layout()
plt.show()

# Résidus de Pearson standardisés vs fitted
plt.figure(figsize=(9,4))
plt.scatter(x_for_plot, pear_std, s=10, alpha=0.5)
plt.axhline(0, color='r', ls='--')
plt.xscale('log')
plt.title("Résidus de Pearson standardisés vs valeurs ajustées (Gamma")
plt.xlabel("Valeurs ajustées ( $\hat{\mu}$ ) [échelle log]")
plt.ylabel("Résidus de Pearson standardisés (studentisés)")
plt.tight_layout()
plt.show()

```





Analyse des résidus du modèle Gamma:

Les résidus de déviance mesurent l'écart entre les valeurs observées et les valeurs ajustées du modèle, en tenant compte de la variance supposée par la loi Gamma. Ils permettent d'évaluer la qualité d'ajustement globale et de détecter d'éventuelles tendances ou biais systématiques. Les résidus de Pearson standardisés (ou studentisés) représentent ces écarts normalisés par la variance et corrigés de l'influence de chaque observation (leverage). Ils sont utilisés pour identifier des observations atypiques ou influentes. Dans notre modèle Gamma, les deux graphiques montrent une dispersion croissante des résidus avec la valeur ajustée, reflet attendu du fait que la variance du Gamma croît avec le carré de la moyenne. Les résidus sont globalement centrés autour de zéro. Quelques valeurs extrêmes traduisent la présence naturelle de sinistres graves, sans remettre en cause la validité du modèle.

Courbe de Lorenz

```
In [ ]: def lorenz_gini_severity_noexpo(res_glm, y_col="ClaimAmount", plot_
        """
        Courbe de Lorenz e pour un modèle de sévérité (GLM Gamma).
        - Tri par sévérité prédite (mu) ; abscisse = proportion cumulée
        - Ne requiert pas la colonne 'Exposure'.
        """
```

```

df_fit = res_glm.model.data.frame.copy()
y = df_fit[y_col].astype(float).to_numpy()
mu_amt = np.asarray(res_glm.fittedvalues, dtype=float)

# Nettoyage
m = np.isfinite(y) & np.isfinite(mu_amt)
y, mu_amt = y[m], mu_amt[m]

if y.size == 0 or np.sum(y) == 0:
    print("Données insuffisantes ou somme des montants observés")
    return {"gini": np.nan, "x": None, "Y": None}

# Tri croissant par sévérité prédite (montant prédit)
order = np.argsort(mu_amt)
y_sorted = y[order]
mu_sorted = mu_amt[order]

# Abscisse : proportion cumulée de la population (égalité des p
n = len(y_sorted)
x = np.arange(1, n+1) / n
Y = np.cumsum(y_sorted) / np.sum(y_sorted)

# Ajouter (0,0) et (1,1)
x = np.concatenate(([0.0], x, [1.0]))
Y = np.concatenate(([0.0], Y, [1.0]))

# Indice de Gini (non pondéré)
try:
    auc = np.trapezoid(Y, x)
except Exception:
    auc = np.trapezoid(Y, x) # fallback
gini = abs(1.0 - 2.0 * auc)
print(f"Indice de Gini sévérité : {gini:.3f}")

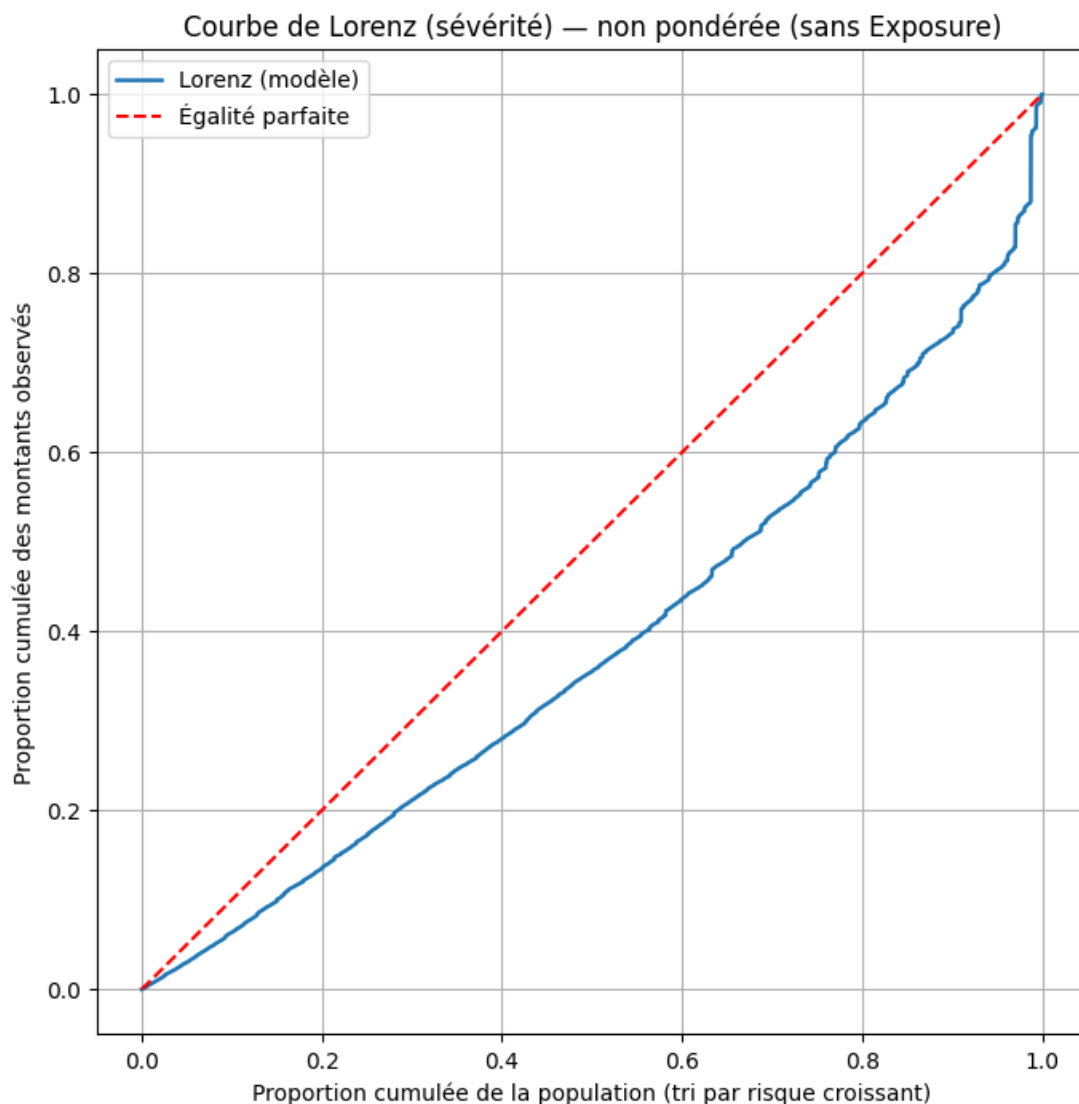
# Tracé
plt.figure(figsize=(8,8))
plt.plot(x, Y, label="Lorenz (modèle)", linewidth=1.8)
plt.plot([0,1], [0,1], "r--", label="Égalité parfaite")
plt.xlabel("Proportion cumulée de la population (tri par risque")
plt.ylabel("Proportion cumulée des montants observés")
plt.title(plot_title or "Courbe de Lorenz – Sévérité ")
plt.legend(); plt.grid(True); plt.show()

return {"gini": gini, "x": x, "Y": Y}

# Application (exemple)
résultats_sev_noexpo = lorenz_gini_severity_noexpo(
    resGamma_final, y_col="ClaimAmount",
    plot_title="Courbe de Lorenz (sévérité) – non pondérée (sans Exp
)

```

Indice de Gini sévérité : 0.234



Validation croisée pour vérifier le surapprentissage

```
In [ ]: def kfold_lorenz_oof_severity_noexpo(res_glm, y_col="ClaimAmount",
                                             k=5, shuffle=True, random_state=None,
                                             plot=True, overlay_insample=False):
    """
    OOF k-fold pour GLM Gamma sans exposure :
    - Refit sur k-1 folds (sans exposure)
    - Prédit sur le fold restant ( $\mu$  par contrat)
    - Lorenz / Gini non pondérés (abscisse = proportion cumulée de la population triée par risque croissant)
    """
    from sklearn.model_selection import KFold

    df_all = res_glm.model.data.frame.copy()
    formula = getattr(res_glm.model, "formula", None)
```

```

family = res_glm.family
if formula is None:
    raise ValueError("Le modèle doit provenir d'un fit via une

oof_mu = pd.Series(index=df_all.index, dtype=float)
kf = KFold(n_splits=k, shuffle=shuffle, random_state=random_sta
gini_folds = []

for i, (tr_idx, va_idx) in enumerate(kf.split(df_all), start=1)
    df_tr = df_all.iloc[tr_idx].copy()
    df_va = df_all.iloc[va_idx].copy()

    # Refit sans exposure
    m_fold = smf.glm(formula=formula, data=df_tr, family=family
    r_fold = m_fold.fit(maxiter=200, disp=False)

    # prédictions ( $\mu$  par contrat)
    mu_va = np.asarray(r_fold.predict(df_va), dtype=float)
    oof_mu.iloc[va_idx] = mu_va

    # Lorenz fold (non pondéré : abscisse = proportion de la po
    y_va = df_va[y_col].astype(float).to_numpy()
    # filtrage sécurité
    ok = np.isfinite(y_va) & np.isfinite(mu_va)
    y_va, mu_va = y_va[ok], mu_va[ok]
    if y_va.size == 0 or y_va.sum() == 0:
        print(f"[CV] Fold {i}/{k} : données insuffisantes")
        gini_folds.append(np.nan)
        continue

    order = np.argsort(mu_va)
    y_ord = y_va[order]
    n = len(y_ord)
    x = np.arange(1, n+1) / n
    Y = np.cumsum(y_ord) / np.sum(y_ord)
    x = np.concatenate(([0.0], x, [1.0]))
    Y = np.concatenate(([0.0], Y, [1.0]))
    try:
        auc = np.trapezoid(Y, x)
    except AttributeError:
        auc = np.trapz(Y, x)
    g_fold = abs(1.0 - 2.0 * auc)
    gini_folds.append(g_fold)
    print(f"[CV] Fold {i}/{k} : Gini OOF (non pondéré) = {g_fold

# Gini OOF global (recolle les prédictions OOF)
df_oof = df_all.copy()
df_oof["mu_oof"] = oof_mu
valid = df_oof["mu_oof"].notna()
if valid.sum() == 0 or df_oof.loc[valid, y_col].sum() == 0:
    print("Pas de prédictions OOF valides ou somme des montants
    return {"gini_folds": gini_folds, "gini_oof": np.nan, "oof_

df_oof = df_oof.loc[valid].copy()
order = np.argsort(df_oof["mu_oof"].to_numpy(dtype=float))
y_o = df_oof[y_col].astype(float).to_numpy()[order]
n_tot = len(y_o)
xo = np.arange(1, n_tot+1) / n_tot
yo = np.cumsum(y_o) / np.sum(y_o)

```

```

xo = np.concatenate(([0.0], xo, [1.0]))
yo = np.concatenate(([0.0], yo, [1.0]))
try:
    auc_o = np.trapezoid(yo, xo)

except AttributeError:
    auc_o = np.trapezoid(yo, xo)
gini_oof = abs(1.0 - 2.0 * auc_o)

print(f"\n[CV] Gini 00F moyen (±sd) : {np.nanmean(gini_folds):.3f} ± {np.nanstd(gini_folds):.3f}")
print(f"[CV] Gini 00F global : {gini_oof:.3f}")

# Tracé 00F global
if plot:
    plt.figure(figsize=(7,7))
    plt.plot(xo, yo, label=f"Lorenz 00F global ({k}-fold)")
    if overlay_insample and 'résultats_sev_noexpo' in globals():
        plt.plot(résultats_sev_noexpo["x"], résultats_sev_noexpo["y"], label="Insample")
    plt.plot([0,1],[0,1], 'r--', label='Égalité parfaite')
    plt.xlabel("Proportion cumulée de la population (tri par risque croissant)")
    plt.ylabel("Proportion cumulée des montants observés")
    plt.title("Lorenz Sévérité – 00F (non pondéré)")
    plt.legend(); plt.grid(True); plt.show()

# Déciles 00F (non pondéré)
df_oof["score"] = df_oof["mu_oof"]
df_oof["decile"] = pd.qcut(df_oof["score"], 10, labels=False, duplicates="drop")
cal = df_oof.groupby("decile").apply(
    lambda g: pd.Series({
        "n": len(g),
        "montants": g[y_col].sum(),
        "sev_obs": g[y_col].mean(),
        "sev_pred": g["mu_oof"].mean()
    })
)
print("\nTable déciles 00F (non pondérée) :")
print(cal.round(4))

return {
    "gini_folds": gini_folds,
    "gini_oof": gini_oof,
    "oof_pred": oof_mu,
    "deciles": cal,
    "x_oof": xo, "y_oof": yo
}

# Lancement CV (00F) – exemple d'appel
cv_sev_noexp = kfold_lorenz_oof_severity_noexpo(
    resGamma_final,
    y_col="ClaimAmount",
    k=5, shuffle=True, random_state=42,
    plot=True, overlay_insample=True
)

```

[CV] Fold 1/5 : Gini 00F (non pondéré) = 0.029
 [CV] Fold 2/5 : Gini 00F (non pondéré) = 0.306
 [CV] Fold 3/5 : Gini 00F (non pondéré) = 0.051
 [CV] Fold 4/5 : Gini 00F (non pondéré) = 0.016
 [CV] Fold 5/5 : Gini 00F (non pondéré) = 0.247

[CV] Gini 00F moyen ($\pm$ sd) : 0.130 $\pm$ 0.136
 [CV] Gini 00F global : 0.149

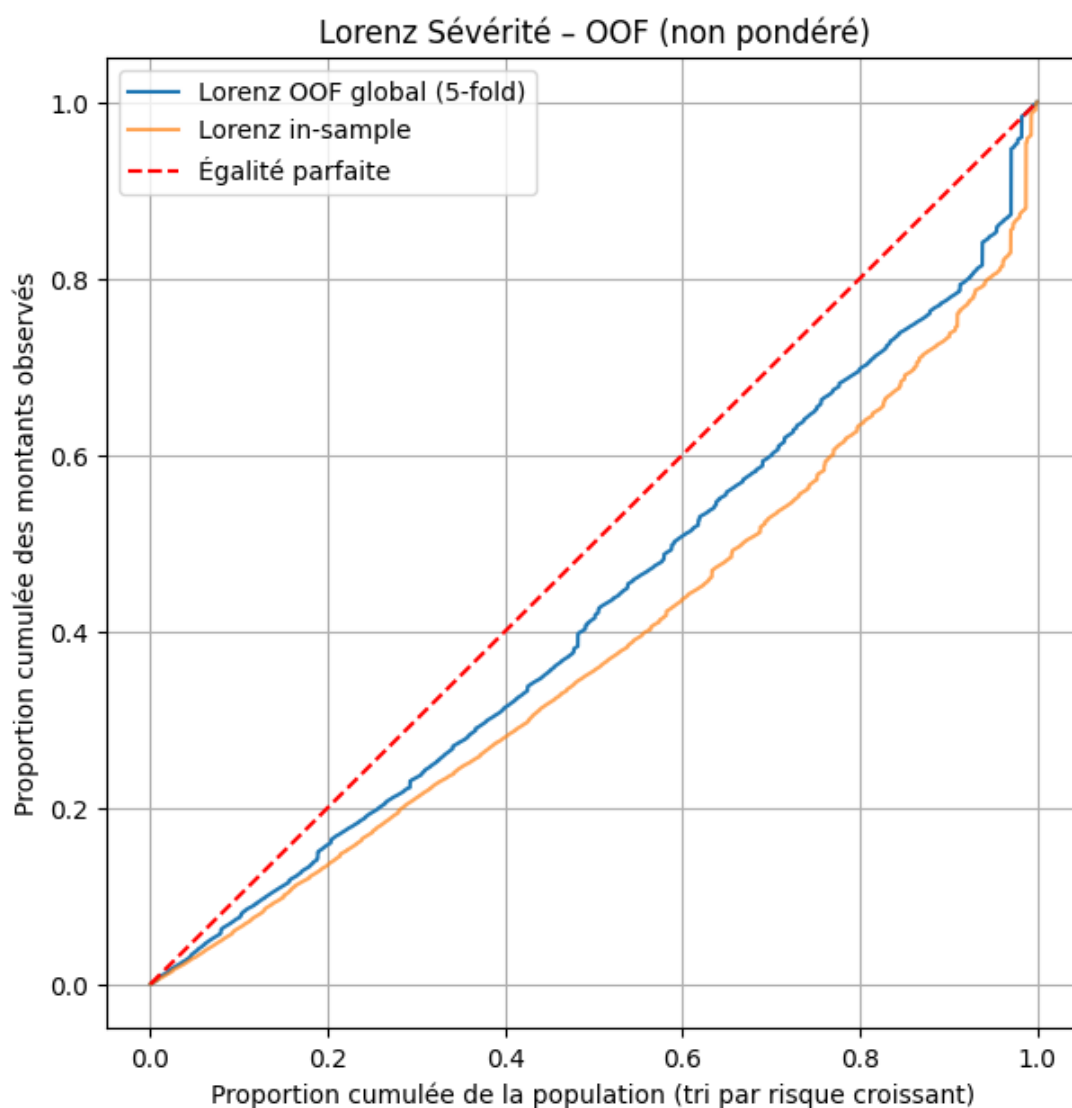


Table déciles 00F (non pondérée) :

	n	montants	sev_obs	sev_pred
decile				
1	2645.0	4136779.31	1563.9997	1235.7030
2	2644.0	4549055.43	1720.5202	1417.6650
3	2645.0	4185463.84	1582.4060	1510.8514
4	2644.0	4354960.79	1647.1107	1577.8702
5	2644.0	5546720.20	2097.8518	1642.1769
6	2644.0	5143409.08	1945.3136	1707.7819
7	2645.0	5004715.66	1892.1420	1782.9837
8	2644.0	5396953.02	2041.2076	1882.9681
9	2644.0	4358482.70	1648.4428	2042.9780
10	2645.0	12261562.47	4635.7514	2501.7809

2924316.py:99: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```
cal = df_oof.groupby("decile").apply(
```

Peu de sur-apprentissage le modèle est conservé tel quel

MODELE PRICING

```
In [ ]: # --- 0) Copies & normalisation douce des IDs ---
df_freq2 = df_freq.copy(deep=True)
df_sev2 = df_sev.copy(deep=True)

def norm_id(s):
    s = pd.Series(s, dtype="object").astype(str).str.strip()
    s = s.str.replace(r"\.0$", "", regex=True) # enlève ".0" final
    s = s.str.lstrip("0") # enlève zéros en t
    s = s.replace("", np.nan)
    return s

df_freq2["IDpol"] = norm_id(df_freq2["IDpol"])
df_sev2["IDpol"] = norm_id(df_sev2["IDpol"])
df_sev2["ClaimAmount"] = pd.to_numeric(df_sev2["ClaimAmount"], erro

# --- 1) Trouver les IDs orphelins (présents en sévérité mais pas en
ids_freq = set(df_freq2["IDpol"].dropna().unique())
sev_by_id = df_sev2.groupby("IDpol", dropna=False)["ClaimAmount"].s
ids_sev = set(sev_by_id.dropna().index)
unmatched_ids = sorted(list(ids_sev - ids_freq))

# Log d'audit (utile si tu veux garder une trace des 6 lignes supprim
if unmatched_ids:
    audit = (df_sev2.loc[df_sev2["IDpol"].isin(unmatched_ids)]
             .groupby("IDpol", as_index=False)
             .agg(total_claim=("ClaimAmount", "sum"), n_rows=("ClaimA
             .sort_values("total_claim", ascending=False))
    audit.to_csv("unmatched_claims_removed.csv", index=False)

print(f"IDs sévérité supprimés (orphelins) : {len(unmatched_ids)}")
if unmatched_ids:
    print("Extrait des IDs supprimés :", unmatched_ids[:6])

# --- 2) Filtrer sévérité sur le périmètre commun (on supprime effe
df_sev_matched = df_sev2.loc[df_sev2["IDpol"].isin(ids_freq)].copy()

# --- 3) Agréger les montants par police et les mapper dans df_freq
claim_by_id = df_sev_matched.groupby("IDpol")["ClaimAmount"].sum()
df_freq2["ClaimAmount"] = df_freq2["IDpol"].map(claim_by_id).fillna

# --- 4) Contrôle de cohérence des totaux sur périmètre commun ---
total_sev_kept = float(df_sev_matched["ClaimAmount"].sum())
total_freq_mapped = float(df_freq2.drop_duplicates("IDpol")["ClaimA

print(f"Total sévérité conservé (match freq) : {total_sev_kept:.2
print(f"Total mappé côté fréquence (unique) : {total_freq_mapped:
```



```

if abs(total_sev_kept - total_freq_mapped) <= 1e-6:
    print("Tout va bien – 6 polices orphelines supprimées, un seul dataset propre")
else:
    print("Écart résiduel – vérifie les IDs ou le périmètre des fichiers")

```

IDs sévérité supprimés (orphelins) : 6
 Extrait des IDs supprimés : ['2220367', '2227533', '2262511', '2277846', '2282134', '2286775']
 Total sévérité conservé (match freq) : 54,938,102.50
 Total mappé côté fréquence (unique) : 54,938,102.50
 Tout va bien – 6 polices orphelines supprimées, un seul dataset propre
 re: df_freq2

```

In [ ]: # =====
# 1) Base propre
# =====
base = df_freq2.copy()
base = base.loc[pd.to_numeric(base["Exposure"], errors="coerce") > 0]
base["ClaimAmount"] = pd.to_numeric(base["ClaimAmount"], errors="coerce")

# =====
# 2) Prédiction avec nos modèles existants
# =====
# Fréquence attendue (Binomiale négative) – avec exposure
base["E_N"] = resBinomNeg.predict(base, exposure=base["Exposure"]).astype(float)

# Sévérité attendue (Gamma, sans exposure)
# (sécuriser les bornes si bs() a été utilisé dans resGamma_final)
train_df = getattr(resGamma_final.model, "data", None)
train_df = train_df.frame if (train_df is not None and hasattr(train_df, "frame")) else train_df
if train_df is not None:
    for col in ["DrivAge", "VehAge"]:
        if col in base.columns and col in train_df.columns:
            mn = pd.to_numeric(train_df[col], errors="coerce").min()
            mx = pd.to_numeric(train_df[col], errors="coerce").max()
            base[col] = pd.to_numeric(base[col], errors="coerce").clip(mn, mx)

base["E_C"] = resGamma_final.predict(base).astype(float)

# =====
# 3) Prime pure prédite & résumé global
# =====
base["S_pred"] = base["E_N"] * base["E_C"]

expo_sum = base["Exposure"].sum()
pp_obs = base["ClaimAmount"].sum() / expo_sum
pp_pred = base["S_pred"].sum() / expo_sum
gap_rel = abs(pp_pred - pp_obs) / pp_obs if pp_obs > 0 else np.nan

print("\n=== Résumé global de la Prime Pure ===")
print(f"Prime pure moyenne observée : {pp_obs:,.2f}")
print(f"Prime pure moyenne prédite : {pp_pred:,.2f}")
print(f"Écart relatif : {100*gap_rel:,.2f}%")

# =====
# 4) Déciles de calibration
# =====
# tri par score = prime pure prédite / exposition
score = (base["S_pred"] / base["Exposure"]).to_numpy()
base["decile"] = pd.qcut(score, 10, labels=["F1", "F2", "F3", "F4", "F5", "F6", "F7", "F8", "F9", "F10"])

```

```

base["decile"] = pd.qcut(score, 10, labels=False, duplicates="drop")
tab = base.groupby("decile").apply(
    lambda g: pd.Series({
        "Exposure": g["Exposure"].sum(),
        "S_obs": g["ClaimAmount"].sum(),
        "S_pred": g["S_pred"].sum(),
        "pp_obs": g["ClaimAmount"].sum() / g["Exposure"].sum(),
        "pp_pred": g["S_pred"].sum() / g["Exposure"].sum(),
    })
)
print("\n=== Calibration par décile (Prime Pure) ===")
print(tab.round(2))

# =====
# 5) Courbe de Lorenz (pondérée expo) & Gini
# - Abscisse = exposition cumulée
# - Ordonnée = sinistres observés cumulés
# - Tri par risque (score) croissant
# =====
def lorenz_gini_weighted(df, amount_col, weight_col, score_col, title):
    """
    Courbe de Lorenz pondérée par l'exposition, triée par score croissant
    - amount_col : montants observés (ex: ClaimAmount)
    - weight_col : poids (Exposure)
    - score_col : score de tri (ex: S_pred / Exposure)
    - also_plot_pred : (optionnel) série de montants 'prédits' pour la courbe
    Retourne (gini, (x, Y_obs), (x, Y_pred) éventuel)
    """
    d = df[[amount_col, weight_col, score_col]].copy()
    d[amount_col] = pd.to_numeric(d[amount_col], errors="coerce").fillna(0)
    d[weight_col] = pd.to_numeric(d[weight_col], errors="coerce").fillna(0)
    d = d.loc[np.isfinite(d[weight_col]) & (d[weight_col] > 0)].copy()

    # tri par score
    d = d.sort_values(score_col, kind="mergesort")
    w = d[weight_col].to_numpy(float)
    y = d[amount_col].to_numpy(float)

    # cumulés pondérés
    x = np.cumsum(w) / w.sum()
    Y = np.cumsum(y) / y.sum() if y.sum() > 0 else np.zeros_like(x)

    # ajouter (0,0) et (1,1)
    x_plot = np.r_[0.0, x, 1.0]
    Y_plot = np.r_[0.0, Y, 1.0]

    # AUC & Gini pour la courbe observée
    try:
        auc = np.trapezoid(Y_plot, x_plot)
    except AttributeError:
        auc = np.trapz(Y_plot, x_plot)
    gini = abs(1.0 - 2.0 * auc)

    # Plot
    plt.figure(figsize=(7,7))
    plt.plot(x_plot, Y_plot, label=f"Observé (Gini={gini:.3f})", linewidth=2)

    # Option : afficher aussi la courbe cumulée des montants prédits
    if also_plot_pred is not None:
        p = d[pred_col].to_numpy(float)
        p = p[p.notnull()]
        p = p[p > 0]
        p = p.sort_values(score_col, kind="mergesort")
        p = p.cumsum() / p.sum()
        p = np.r_[0.0, p, 1.0]
        plt.plot(x_plot, p, label=f"Prédit (Gini={gini:.3f})", linewidth=2, style="dashed")

```

```

d["_pred"] = pd.to_numeric(also_plot_pred, errors="coerce")
yp = d["_pred"].to_numpy(float)
Yp = np.cumsum(yp) / yp.sum() if yp.sum() > 0 else np.zeros(yp.size)
Yp_plot = np.r_[0.0, Yp, 1.0]
plt.plot(x_plot, Yp_plot, linestyle="--", label="Prédit (cumulé)")

plt.plot([0,1],[0,1],"r--",label="Égalité parfaite")
plt.xlabel("Proportion cumulée d'exposition (triée par risque croissant)")
plt.ylabel("Proportion cumulée des montants observés")
plt.title(title or "Courbe de Lorenz – Prime pure (pondérée exposée)")
plt.legend(); plt.grid(True); plt.tight_layout(); plt.show()

return gini, (x_plot, Y_plot)

# score de tri = S_pred / Exposure
base["score_pp"] = base["S_pred"] / base["Exposure"]

# Courbe de Lorenz & Gini (observé, tri par score)
gini_pp, (x_lor, y_lor) = lorenz_gini_weighted(
    base,
    amount_col="ClaimAmount",
    weight_col="Exposure",
    score_col="score_pp",
    title="Lorenz pondérée – Prime pure (tri par risque prédit)",
    also_plot_pred=base["S_pred"] # affiche aussi la courbe cumulée
)

print(f"\nIndice de Gini (prime pure, pondéré expo, trié par risque")

```

=== Résumé global de la Prime Pure ===

Prime pure moyenne observée : 153.24

Prime pure moyenne prédite : 177.63

Écart relatif : 15.91%

=== Calibration par décile (Prime Pure) ===

	Exposure	S_obs	S_pred	pp_obs	pp_pred
decile					
1	38223.17	2247161.25	3025406.54	58.79	79.15
2	39440.74	3255224.71	3968792.49	82.53	100.63
3	39818.86	3162299.90	4531065.55	79.42	113.79
4	39557.00	4165263.69	4983908.94	105.30	125.99
5	39694.03	3938813.44	5494358.26	99.23	138.42
6	39777.00	3938813.44	5494358.26	99.23	138.42
7	39777.00	3938813.44	5494358.26	99.23	138.42
8	39777.00	3938813.44	5494358.26	99.23	138.42
9	39777.00	3938813.44	5494358.26	99.23	138.42
10	39777.00	3938813.44	5494358.26	99.23	138.42

6	39047.60	5261942.48	5942138.17	134.76	152.18
7	37556.38	5087321.17	6448057.26	135.46	171.69
8	34843.67	6749233.35	7212055.43	193.70	206.98
9	28214.12	7737620.87	8543035.30	274.25	302.79
10	22103.87	13333221.64	13532264.49	603.21	612.21

```

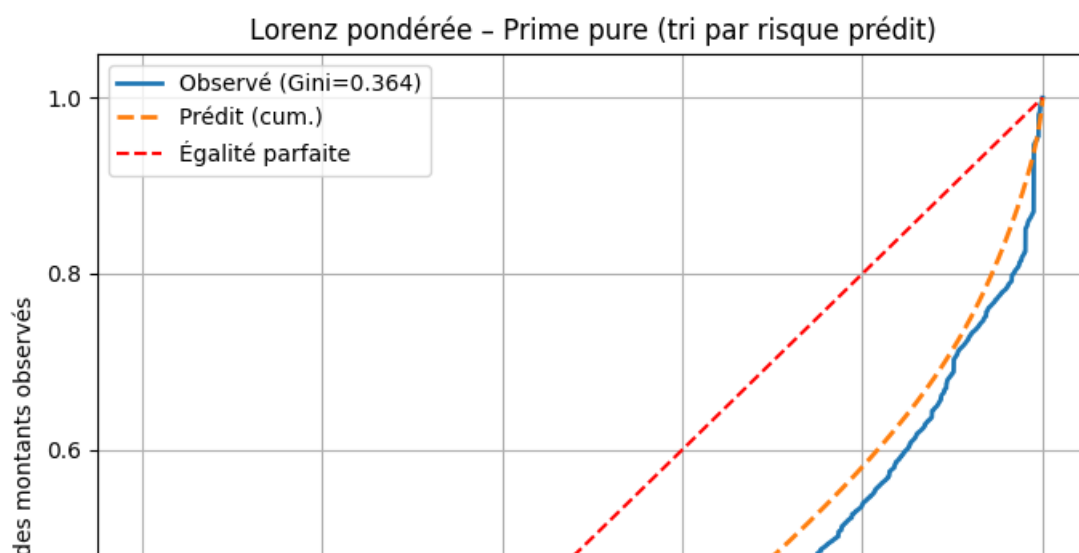
/var/folders/61/k5ckz_dn2hs4k4v4pjww68p80000gn/T/ipykernel_66303/107
9128282.py:48: FutureWarning: DataFrameGroupBy.apply operated on the
grouping columns. This behavior is deprecated, and in a future versi
on of pandas the grouping columns will be excluded from the operatio
n. Either pass `include_groups=False` to exclude the groupings or ex
plicitly select the grouping columns after groupby to silence this w
arning.

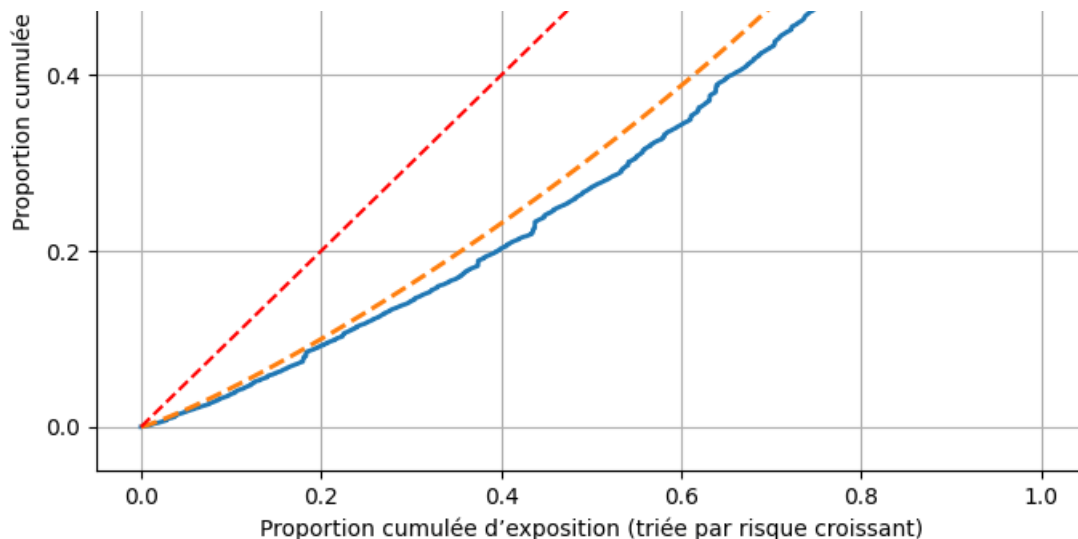
```

```

    tab = base.groupby("decile").apply(

```





Indice de Gini (prime pure, pondéré expo, trié par risque) : 0.364

L'indice de Gini est de 0,364. Le modèle final est satisfaisant et pourra être utilisé pour la tarification.

```
In [ ]: # ===== SIMULER UN INDIVIDU & CALCULER SA PRIME PURE =====

# Colonnes nécessaires aux deux modèles (adapte si ton modèle diffère)
CAT_VARS = ["VehBrand", "VehGas", "Area", "Region"]
NUM_VARS = ["DrivAge", "VehAge", "BonusMalus", "VehPower", "Density"]

def sample_like(series):
    """Échantillonne selon la distribution empirique (catégorielle ou numérique)"""
    s = series.dropna()
    if s.empty:
        return np.nan
    # catégorielle: tirage par fréquences ; numérique: bootstrap simple
    if s.dtype == "O" or str(s.dtype).startswith("category"):
        probs = s.value_counts(normalize=True)
        return np.random.choice(probs.index.to_list(), p=probs.values)
    else:
        return s.sample(1, replace=True).iloc[0]

def simulate_one_contract(df_source: pd.DataFrame, seed=None):
    """Crée 1 individu réaliste en échantillonnant chaque variable"""
    rng = np.random.default_rng(seed)
    row = {}
    # Catégorielles
    for c in CAT_VARS:
        if c in df_source.columns:
            row[c] = sample_like(df_source[c])
    # Numériques
    for c in NUM_VARS:
        if c in df_source.columns:
            row[c] = float(sample_like(pd.to_numeric(df_source[c], errors="coerce")))
    # Exposure > 0
    if "Exposure" in row and (pd.isna(row["Exposure"]) or row["Exposure"] == 0):
        # valeur par défaut: médiane des expositions > 0
        expos = pd.to_numeric(df_source["Exposure"], errors="coerce").median()
        row["Exposure"] = expos
```

```

        row["Exposure"] = float(expos[expos > 0].median())
    return pd.DataFrame([row])

def pure_premium_for_row(x1: pd.DataFrame):
    """
    Calcule E[N], E[C], prime pure = E[N]*E[C] pour un DataFrame d'individus
    en cohérence avec les modèles ajustés.
    """
    # Sûr-typage pour patsy/statsmodels
    for c in ["DrivAge", "VehAge", "BonusMalus", "VehPower", "Density", "Exposure"]:
        if c in x1.columns:
            x1[c] = pd.to_numeric(x1[c], errors="coerce")
    # Fréquence (comptes attendus)
    EN = float(resBinomNeg.predict(x1, exposure=x1["Exposure"]))
    # Sévérité (montant moyen conditionnel)
    EC = float(resGamma_final.predict(x1))
    return EN, EC, EN*EC

# ----- Exemple d'utilisation -----
# 1) Simuler un individu
indiv = simulate_one_contract(df_freq, seed=123)
print("Individu simulé :")
print(indiv.T)

# 2) Calculer sa prime pure
EN, EC, PP = pure_premium_for_row(indiv)
print(f"\nFréquence attendue E[N] : {EN:,.4f}")
print(f"Sévérité attendue E[C] : {EC:,.2f}")
print(f"Prime pure E[S] : {PP:,.2f}")

```

Individu simulé :

	0
VehBrand	B12
VehGas	Regular
Area	C
Region	Languedoc-Roussillon
DrivAge	55.0
VehAge	17.0
BonusMalus	50.0
VehPower	11.0
Density	884.0
Exposure	0.87

Fréquence attendue E[N] : 0.0570
 Sévérité attendue E[C] : 1,202.43
 Prime pure E[S] : 68.51

```

/var/folders/61/k5ckz_dn2hs4k4v4pjww68p80000gn/T/ipykernel_66303/374
6753973.py:51: FutureWarning: Calling float on a single element Series
is deprecated and will raise a TypeError in the future. Use float(
ser.iloc[0]) instead
    EN = float(resBinomNeg.predict(x1, exposure=x1["Exposure"]))
/var/folders/61/k5ckz_dn2hs4k4v4piww68p80000gn/T/ipykernel_66303/374

```

```
6753973.py:53: FutureWarning: Calling float on a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.iloc[0]) instead
    EC = float(resGamma_final.predict(x1))
```